Kiến trúc VNOJ

Phân tích từ source của `VNOI-Admin/OJ` (site) và `VNOI-Admin/judge-server`
Bản đầy đủ

11/11/2026

Contents

1 Phạm vi, nguồn và cách đọc **2**
1.1 Độ mới của source . . . 2
1.2 Cách đọc các nhãn độ chắc chắn . . . 3

2 Bức tranh tổng thể **3**
2.1 Danh mục tiến trình . . . 4
2.2 Ai nói chuyện với ai . . . 5

3 Site (`VNOI-Admin/OJ`) chi tiết **5**
3.1 Bản đồ mã nguồn . . . 5
3.2 Lớp web . . . 6
3.3 Lớp dữ liệu . . . 6
3.4 `bridged`: bộ điều phối chấm bài . . . 7
3.4.1 Cấu trúc tiến trình . . . 7
3.4.2 Khung gói tin . . . 8
3.4.3 Giao thức Django ↔ `bridged` . . . 8
3.4.4 Giao thức judge ↔ `bridged` . . . 9
3.4.5 Máy trạng thái của một bài nộp . . . 9
3.4.6 Lập lịch trong `JudgeList` . . . 10
3.4.7 Tính điểm và kết luận khi `grading-end` . . . 10
3.4.8 Hành vi khi có sự cố . . . 10
3.5 Hệ thống sự kiện thời gian thực . . . 11
3.6 Celery . . . 11
3.7 Kho test data và đường đi của bài tập . . . 12
3.8 Balancer (tuỳ chọn) . . . 13
3.9 Dịch vụ ngoài . . . 13

4 Judge server (`VNOI-Admin/judge-server`) chi tiết **14**
4.1 Bản đồ mã nguồn . . . 14
4.2 Mô hình tiến trình . . . 14
4.3 Cấu hình và khởi động . . . 15
4.4 Một bài nộp bên trong judge . . . 15
4.5 Định dạng bài (`init.yml`) . . . 16
4.6 Grader: cách chấm một test . . . 17

4.7 Checker và các quy ước mã thoát . . . 17
4.8 Executor: một lớp cho mỗi ngôn ngữ . . . 17
4.9 Sandbox `cptbox` . . . 18
4.10 Đóng gói: các "tier" image . . . 19
4.11 Điều khiển và quan sát . . . 19

5 Vòng đời một bài nộp từ đầu đến cuối 19

6 Từng thành phần: vì sao cần, thiếu thì sao, thay hoặc bỏ được không 21
6.1 Bảng quyết định tổng hợp . . . 25

7 Sự cố, bảo mật và các phát hiện từ việc đọc code 26
7.1 Ma trận sự cố . . . 26
7.2 Bảo mật và cấu hình mặc định . . . 26
7.3 Các phát hiện . . . 27
7.4 Những điều chưa xác minh . . . 28

A Phụ lục 28
A.1 Thuật ngữ và mã trạng thái . . . 28
A.2 Cách kiểm tra lại các khẳng định . . . 28
A.3 Các file đáng đọc nhất . . . 29

1 Phạm vi, nguồn và cách đọc

Tài liệu này mô tả **kiến trúc** của VNOJ: hệ thống gồm những khối nào, mỗi khối làm gì bên trong, các khối nói chuyện với nhau bằng giao thức gì, thiếu một khối thì chuyện gì xảy ra, và khối nào thay hoặc bỏ được. Nó dựa **hoàn toàn vào việc đọc source** của hai repo mà VNOI đang phát triển:

- `VNOI-Admin/OJ`: site Django (fork của DMOJ `online-judge`), kèm daemon `bridged`, balancer, daemon WebSocket và các tác vụ Celery.
- `VNOI-Admin/judge-server`: máy chấm (fork của DMOJ `judge-server`), kèm sandbox `cptbox`, các executor và checker.

1.1 Độ mới của source

Tôi kiểm tra ngày commit của từng repo trước khi đọc, và chỉ dùng hai repo ở dòng đầu làm căn cứ.

Repo / nhánh	Commit mới nhất	Dùng làm căn cứ?	Ghi chú
`OJ`, nhánh `master`	`209a32e`, 04/11/2026	**Có**	Đã có nhiều kho lưu trữ (filesystem/S3), judge protocol v2, tier của judge, `beat_schedule`, OAuth, API đồng bộ kỳ thi.
`judge-server`, nhánh `master`	`23759617`, 10/10/2026	**Có**	Bản clone mới, đặt ở `vnoj-src/judge-server-upstream`.
`vnoj-docker`	13/09/2023	Không	Cũ hơn site ∼3 năm (không có S3, không có celery beat).
`vnoj-docs`	19/09/2024	Không	Cũ hơn site ∼2 năm.

Các nhánh của `judge-server` đi trước `master`. Tôi đã mở diff của những nhánh còn hoạt động. Chúng **chưa merge**, nên không được tính là kiến trúc hiện hành, nhưng cho thấy hướng phát triển:

Nhánh	Commit mới nhất	Nội dung
`thuc/ce-msg`	08/11/2026	1 commit: giải thích lỗi biên dịch "File too large" (3 file). Không đổi kiến trúc.
`hiep/landlock`	11/10/2026	6 commit: thay một phần kiểm tra file bằng Landlock (không còn bắt ptrace cho file), I/O qua `memfd`.
`feat/microvm-api-judge`	11/10/2026	1 commit: kho test data S3 **phía judge** và chế độ judge qua HTTP API.
`wip/parallel-case-grading`	10/10/2026	WIP: chấm các test case của một bài nộp song song.
`hiep/ptraceless`, `fpt26`, `ai`, …	2025–2026	Thử nghiệm sandbox không ptrace; thêm PyTorch; nhánh cũ.

Hệ quả quan trọng: trong `master`, **judge không có mã nào tải test data từ S3** (tìm `s3`, `boto` trong `dmoj/` chỉ ra hai kết quả trong file test, không phải mã chạy). Mã S3 phía judge nằm ở nhánh chưa merge `feat/microvm-api-judge`.

Lưu ý về thư mục cục bộ. Thư mục `/home/worker/AlgoJudge/judge-server` hiện không còn là bản clone của `VNOI-Admin/judge-server`: `origin` của nó là `Algorit-loop/AlgoJudge`, chỉ có 1 commit ("first update") và không có `.docker/tiervnoj`. Tôi **không dùng** thư mục đó làm căn cứ và không sửa gì trong đó.

1.2 Cách đọc các nhãn độ chắc chắn

Mỗi khẳng định kỹ thuật quan trọng mang một nhãn:

[S] **Đã đọc trực tiếp trong source**; có thể mở file và kiểm tra. Tên file đi kèm trong dấu ngoặc hoặc cột "Căn cứ".

[I] **Suy ra từ code** (ví dụ từ thứ tự lệnh hay luồng điều khiển), **chưa chạy thử**.

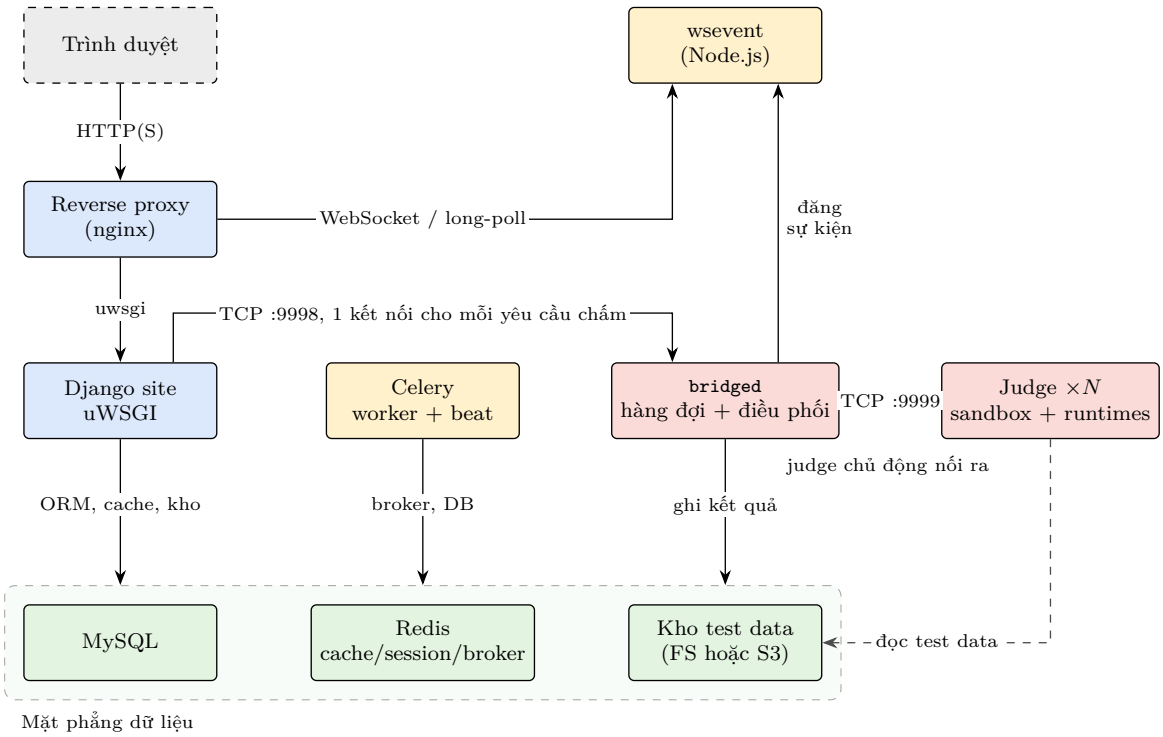
[?] **Chưa xác minh được**; cần quan sát hệ thống chạy thật hoặc hỏi người vận hành.

Giới hạn. Trang `oj.vnoi.info` nằm sau Cloudflare challenge (HTTP 403, `cf-mitigated: challenge`) nên tôi **không quan sát được hạ tầng chạy thật**: số máy chấm, nơi đặt từng tiến trình, CDN, cách đồng bộ test data lên máy chấm. Mọi kết luận là về *thiết kế trong mã nguồn*, không phải về cách VNOI đang vận hành.

2 Bức tranh tổng thể

VNOJ chia được thành bốn mặt phẳng: *web* (phục vụ người dùng), *dữ liệu*, *bất đồng bộ / thời gian thực*, và *chấm bài*. Điều cần nhớ nhất:

> **Django không tự chấm bài.** Nó ghi bài nộp vào DB rồi nhờ daemon `bridged` điều phối. Các máy chấm *tự kết nối ra* `bridged`. Kết quả chấm đi ngược lại qua `bridged`, và **chính** `bridged` tính điểm rồi ghi vào DB. [S]



Hình 1. Kiến trúc tổng thể. Ba mũi tên đi xuống mặt phẳng dữ liệu: Django dùng MySQL, Redis (cache, session) và kho test data; Celery dùng Redis làm broker và ghi MySQL; `bridged` ghi kết quả vào MySQL. Nét đứt: judge đọc test data trực tiếp từ thư mục đã khai báo, không tải qua mạng (mục 3.7). Balancer là phần tuỳ chọn nên vẽ riêng ở Hình 5.

2.1 Danh mục tiến trình

Mỗi hộp trong Hình 1 là một tiến trình (hoặc nhóm tiến trình) riêng. Bảng sau liệt kê cách khởi chạy theo đúng file trong repo:

Tiến trình	Điểm vào trong source	Cổng mặc định	Bắt buộc?
Django (web)	`dmoj/wsgi.py` (uWSGI có trong `requirements.txt`)	do proxy quyết định	**Có.**
`bridged`	`python manage.py runbridged` (`judge/management/commands/runbridged.py`); hoặc `dmoj_bridge_async.py` (patch `gevent`)	9998 (Django), 9999 (judge)	**Có, nếu muốn chấm bài.**
Judge	`dmoj <host> [tên] [khoá]` (`dmoj/judge.py:main`); cổng nối mặc định 9999	(nối ra)	**Có, ít nhất 1.**
Celery worker	module `dmoj_celery.py`, cấu hình ở `dmoj/celery.py`	—	Không để chấm; cần cho rejudge hàng loạt, MOSS, …
Celery beat	lịch `beat_schedule` trong `dmoj/celery.py`	—	Chỉ cho các việc định kỳ.
`wsevent`	`node websocket/daemon.js`; đọc `websocket/config.js` (**không có trong repo**, bị `.gitignore`)	`EVENT_DAEMON_POST` 9997, `EVENT_DAEMON_GET` 9996 (mặc định trong settings)	Không.
Balancer	`python manage.py runbalancer -c cfg.yml`	8888 (cho judge)	Không.

[S] Mọi giá trị trong cột "Cổng mặc định" lấy từ `dmoj/settings.py`: `BRIDGED_JUDGE_ADDRESS`, `BRIDGED_DJANGO_ADDRESS`, `BALANCER_JUDGE_ADDRESS`, `EVENT_DAEMON_POST`, `EVENT_DAEMON_GET`. Cổng thật của `wsevent` do `websocket/config.js` quyết định [?].

Hai điểm vào dùng `gevent` (`dmoj_bridge_async.py`, `dmoj/wsgi_async.py`) tồn tại, nhưng `gevent` **không** nằm trong `requirements.txt`. Cách chạy mặc định của `bridged` là `runbridged`, dùng luồng (thread) thường. [S]

2.2 Ai nói chuyện với ai

Kết nối	Giao thức	Chi tiết
Django → `bridged`	TCP, JSON nén zlib, có tiền tố độ dài 4 byte	**Một kết nối cho mỗi yêu cầu**: kết nối, gửi 1 gói, nhận 1 gói trả lời, đóng (`judge/judgeapi.py:judge_request`). [S]
Judge → `bridged`	Cùng khung gói, kết nối **dài**	Judge nối ra, bắt tay (`handshake`, `version` 2), rồi nhận `submission-request` và gửi kết quả. `bridged` `ping` mỗi 10 giây. [S]
`bridged`/Django → `wsevent`	WebSocket (hoặc AMQP)	`event.post(kênh, thông điệp)`. Mặc định **tắt** (`EVENT_DAEMON_USE = False`). [S]
Trình duyệt ↔ `wsevent`	WebSocket, dự phòng long-poll	Lọc theo kênh, tối đa 5 kênh mỗi kết nối (`websocket/daemon.js`). [S]
Django, Celery → MySQL	Django ORM	Cả `bridged` cũng ghi trực tiếp bằng ORM. [S]
Django, Celery → Redis	Django cache API; Celery broker	Code chỉ dùng API cache chuẩn của Django, không gọi Redis trực tiếp (`grep redis` trong `judge/` không có kết quả mã). [S]
Judge → site (HTTP)	`requests.get`	Chỉ cho ngôn ngữ `file_only` (`OUTPUT`, `SCRATCH`): judge tải nguồn bài nộp từ `SITE_FULL_URL` (`dmoj/utils/helper_files.py:download_source_code`). [S]
Django → dịch vụ ngoài	HTTP(S)	pdfoid, mathoid/texoid, camo, dịch vụ lưu trữ bài, Discord webhook, MOSS, mailgun. Xem mục 3.9. [S]

Hai chi tiết thường bị bỏ qua: (1) **judge không hoàn toàn "chỉ nối ra"**: với bài nộp dạng file, nó cần đường HTTP tới site; (2) `bridged` **không dùng TLS**: nó là `socketserver.TCPServer` thường (`judge/bridge/server.py`). Judge có tuỳ chọn `--secure` để dùng TLS, nên phía `bridged` phải đặt một proxy giải mã TLS ở trước. `bridged` hiểu giao thức PROXY (`BRIDGED_JUDGE_PROXIES`, `judge/bridge/base_handler.py`) để lấy lại IP thật của judge. [S]

3 Site (`VNOI-Admin/OJ`) chi tiết

3.1 Bản đồ mã nguồn

Site là một project Django với app chính tên `judge`. Các thư mục ở gốc repo:

Thư mục / file	Vai trò
`dmoj/`	`settings.py` (856 dòng, mọi cấu hình mặc định), `urls.py`, `celery.py`, `wsgi.py`, `wsgi_async.py`, hai *logging handler* có giới hạn tần suất (`throttle_mail`, `throttle_discord_webhook`).
`judge/models/`	Mô hình dữ liệu: `problem`, `problem_data`, `submission`, `contest`, `profile` (gồm `Organization`, quota), `runtime` (`Judge`, `Language`, `RuntimeVersion`), `comment`, `ticket`, `interface` (blog, thanh điều hướng, `MiscConfig`), `notification`.
`judge/views/`	∼35 module view (bài, kỳ thi, nộp bài, tổ chức, ticket, trạng thái judge, …) và `views/api/api_v2.py`.
`judge/admin/`	Giao diện admin của Django cho các model trên.
`judge/bridge/`	**Daemon** `bridged`: `daemon`, `server`, `base_handler`, `django_handler`, `judge_handler`, `judge_list`.
`judge/balancer/`	Balancer tuỳ chọn (nhiều site dùng chung một dàn judge).
`judge/judgeapi.py`, `judge_priority.py`	Phía Django của giao thức tới `bridged`; bốn mức ưu tiên.
`judge/tasks/`	Tác vụ Celery. `judge/event_poster*.py`: đăng sự kiện thời gian thực.
`judge/contest_format/`	Luật tính điểm kỳ thi: `default`, `ioi`, `legacy_ioi`, `icpc`, `atcoder`, `ecoo`, `vnoj` (đăng ký ở `registry.py`).
`judge/utils/`	Kho dữ liệu bài (`problem_data_storage`, `storages`), biên dịch `init.yml` (`problem_data`), client dịch vụ ngoài (`pdfoid`, `mathoid`, `texoid`, `camo`, `recaptcha`, `pwned`), `problem_archive`, …
`judge/ratings.py`, `performance_points.py`	Hệ số rating và điểm hiệu năng (PP) của người dùng.
`judge/management/commands/`	`runbridged`, `runbalancer`, `addjudge`, `runmoss`, `create_problem`, `import_polygon_package`, `submit_polygon_solutions`, `render_pdf`, `adduser`, `batchadduser`, …
`templates/`, `resources/`, `judge/jinja2/`	Giao diện: template **Jinja2** (không phải template Django mặc định), SCSS/JS, bộ lọc Jinja.
`django_ace/`, `martor/`, `urlshortener/`	Trình soạn thảo Ace, trình soạn Markdown, và một app rút gọn link riêng.
`websocket/`	Daemon `wsevent` bằng Node.js (`daemon.js`, `queue.js`).

3.2 Lớp web

Cấu hình và middleware. `MIDDLEWARE` gồm 19 lớp (`dmoj/settings.py:475`). Những lớp đáng chú ý: `ShortCircuitMiddleware` (cho view đánh dấu `short_circuit_middleware`, trả lời không qua phần còn lại), `DMOJLoginMiddleware` (ép đăng nhập 2FA và đổi mật khẩu bị lộ), `ContestMiddleware` (đưa người dùng đang thi vào ngữ cảnh kỳ thi), `ImpersonateMiddleware`. [S]

Xác thực. Backend: Google OAuth2, Facebook OAuth2, GitHub (chỉ nhận email đã xác minh), `ModelBackend`, và `IPBasedAuthBackend`. Hai lớp 2FA: TOTP và WebAuthn. Site còn là *nhà cung cấp OAuth2* (`django-oauth-toolkit`, `VNOIOAuthValidator`) và có API token cá nhân. Có cả *API đồng bộ kỳ thi* (`/api/v2/sync/...`), **tắt mặc định** (`VNOJ_ENABLE_SYNC_API = False`), bảo vệ bằng `GLOBAL_API_KEY`. [S]

Giao diện. Template Jinja2 qua `django_jinja`; tĩnh qua `django_compressor`; `make_style.sh` dùng `sass` và `postcss` để sinh hai bộ CSS (sáng và tối) từ `resources/`. Trình soạn mã là Ace, trình soạn Markdown là martor, toán dùng MathJax hoặc dịch vụ mathoid/texoid. [S]

Chế độ vận hành đặc biệt. `VNOJ_LOW_POWER_MODE` giới hạn số trang submission và vẽ heat map khi tải cao; `VNOJ_OFFICIAL_CONTEST_MODE` ghi IP khi nộp bài và không ép đổi mật khẩu. [S]

Nộp bài. Luồng nằm trong `judge/views/problem.py:handle_submission_post`: kiểm tra giới hạn (`DMOJ_SUBMISSION_LIMIT = 2` bài chưa chấm xong trừ người có quyền `spam_submission`), ngôn ngữ được phép, người bị cấm, số lần nộp tối đa, *credit* của tổ chức; rồi trong một `transaction.atomic()` tạo `Submission`, `ContestSubmission` (nếu đang thi), `SubmissionSource`; sau cùng gọi `new_submission.judge(force_judge=True, ...)`. Chỉ **sau** khi giao dịch DB kết thúc mới gửi yêu cầu tới `bridged`. [S]

3.3 Lớp dữ liệu

Mô hình chính (rút gọn):

Thực thể	Quan hệ và ghi chú
`Submission`	FK tới `Profile`, `Problem`, `Language`, `Judge` (`judged_on`), `Contest` (`contest_object`). Trạng thái `status` ∈ {QU, P, G, D, IE, CE, AB}, kết quả `result` ∈ {AC, PAC, WA, TLE, MLE, OLE, IR, RTE, CE, IE, SC, AB}. Nhiều chỉ mục kép cho trang danh sách/xếp hạng (`models/submission.py`). [S]
`SubmissionSource`	1–1 với `Submission`; mã nguồn (tối đa 65 536 ký tự). Tách bảng để bảng `Submission` nhẹ. [S]
`SubmissionTestCase`	Nhiều dòng cho mỗi `Submission`; `unique(submission, case)`; có `status`, `time`, `memory`, `points`, `total`, `batch`, `feedback`, `output`. Khi chấm lại, toàn bộ bị xoá. [S]
`Problem`, `ProblemData`, `ProblemTestCase`	`Problem.code` là *id* mà judge dùng (`problem-id`); `Problem.storage` chọn kho; `ProblemData` giữ file zip, generator, checker, grader tuỳ chỉnh; `ProblemTestCase` là từng dòng cấu hình test. [S]
`LanguageLimit`	Giới hạn thời gian/bộ nhớ riêng cho từng cặp (bài, ngôn ngữ); `bridged` đọc khi giao việc. [S]
`Contest`, `ContestProblem`, `ContestParticipation`, `ContestSubmission`	`ContestSubmission` nối bài nộp với bài trong kỳ thi và lượt tham gia. `Contest.banned_judges` cho phép loại judge khỏi kỳ thi. [S]
`Judge`, `RuntimeVersion`, `Language`	`Judge`: `name`, `auth_key` (lưu **dạng thô**), `is_blocked`, `is_disabled`, `tier`, `online`, `ping`, `load`, `storages` (JSON), `runtimes` (M2M tới `Language`). Bảng này do `bridged` cập nhật khi judge nối vào. [S]

Cơ sở dữ liệu. Mặc định của `settings.py` là **SQLite**, nhưng CI chạy trên **MySQL 8.0** (`.github/workflows/build.yml`) và `requirements.txt` có `mysqlclient`. Code có phần dành riêng cho MySQL: `judge/dblock.py` dùng `LOCK TABLES`, `judge/fulltext.py` dùng `MATCH ... AGAINST`, `performance_points.py` chọn `STRAIGHT_JOIN` khi `connection.vendor == 'mysql'`. Vì vậy MySQL/MariaDB là lựa chọn thực tế. [S]

Cache và session. `CACHES = {}` trong `settings.py`, nghĩa là phải đặt trong `local_settings.py`. Session là `cached_db`. Có 24 module trong `judge/` import API cache chuẩn của Django, và tín hiệu (`judge/signals.py`) *xoá cache theo sự kiện* (sửa bài thì xoá fragment HTML, PDF đã render, metadata). Muốn xoá đúng khi có nhiều tiến trình thì cache phải *dùng chung*; `LocMemCache` (mặc định của Django, và là thứ CI dùng) không đáp ứng. [S] [I]

Kho test data. Xem mục 3.7.

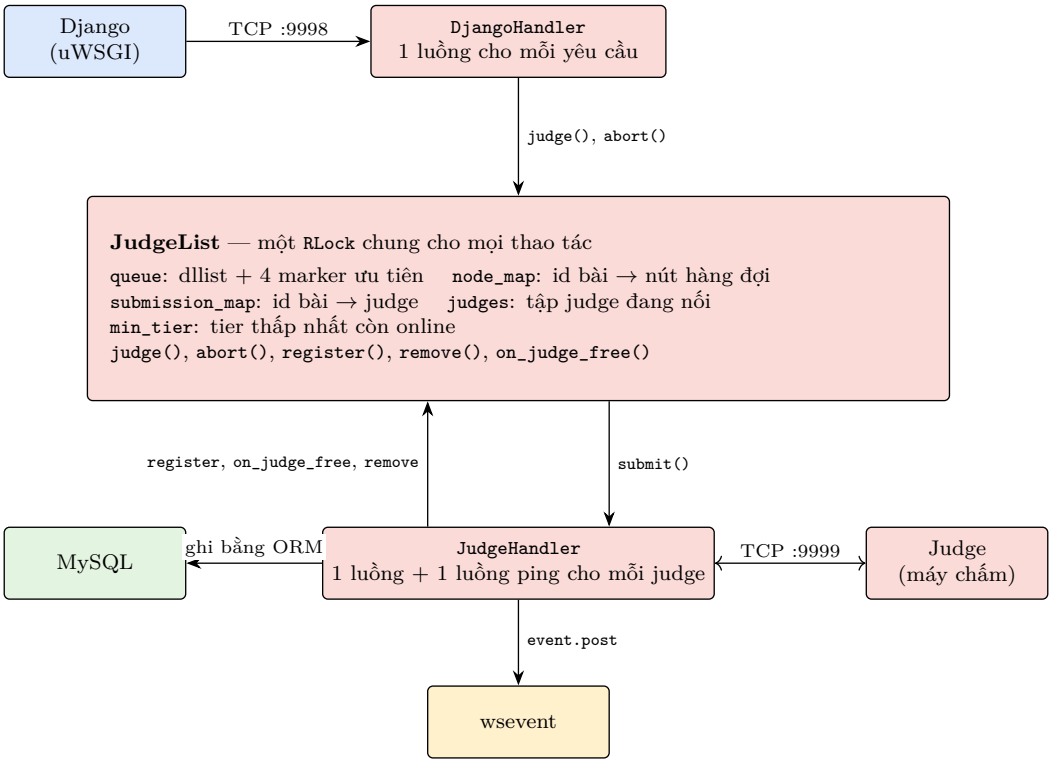
3.4 `bridged`: bộ điều phối chấm bài

Đây là phần quan trọng nhất của kiến trúc nên được đọc kỹ nhất. Code ở `judge/bridge/` (khoảng 1 050 dòng).

3.4.1 Cấu trúc tiến trình

`judge_daemon()` (`daemon.py`) làm bốn việc theo thứ tự: (1) `reset_judges()`: đặt mọi `Judge` thành offline; (2) **đổi mọi bài nộp đang ở trạng thái `QU`, `P`, `G` thành `IE`** (`Submission.IN_PROGRESS_GRADING_STATUS`); (3) tạo một `JudgeList` duy nhất; (4) mở hai server TCP chạy trong hai luồng: server cho Django (`BRIDGED_DJANGO_ADDRESS`, 9998) và server cho judge (`BRIDGED_JUDGE_ADDRESS`, 9999). Mỗi server là `ThreadingMixIn + TCPServer` nên **mỗi kết nối được một luồng riêng**. Tín hiệu `SIGINT/SIGQUIT/SIGTERM` làm daemon tắt. [S]

Bước (2) có hệ quả: **hàng đợi của `bridged` chỉ nằm trong RAM**. Khởi động lại `bridged` là bỏ toàn bộ bài đang chờ và đang chấm (đặt `IE`); người dùng phải nộp lại hoặc admin rejudge. [S]



Hình 2. Bên trong `bridged`. Mọi thay đổi trạng thái hàng đợi và tập judge đều đi qua một `RLock` duy nhất trong `JudgeList`. `DjangoHandler` chỉ đẩy việc vào `JudgeList`; `JudgeHandler` là nơi ghi kết quả vào MySQL và đăng sự kiện.

3.4.2 Khung gói tin

Cả hai phía dùng cùng một khung (`base_handler.py`): *4 byte độ dài (big-endian) + JSON UTF-8 nén zlib.* Kích thước gói tối đa 8 MiB; vượt mức này thì bị ngắt. Có sẵn xử lý giao thức PROXY nếu IP nguồn nằm trong `BRIDGED_JUDGE_PROXIES`. [S]

3.4.3 Giao thức Django ↔ `bridged`

`judge/judgeapi.py` (phía Django) và `django_handler.py` (phía bridged). Mỗi yêu cầu mở một kết nối mới, gửi một gói, đọc một gói trả lời, rồi đóng (`DjangoHandler.on_packet` luôn `raise Disconnect()` sau khi trả lời). [S]

Yêu cầu	Trường chính	Trả lời / tác dụng
`submission-request`	`submission-id`, `problem-id` (= `Problem.code`), `storage`, `language` (khoá), `source`, `judge-id`, `banned-judges`, `priority` (0–3)	`submission-received`. Nếu `priority` ngoài khoảng: `bad-request`.
`terminate-submission`	`submission-id`	`judge-aborted`: `true` nếu đã báo judge dừng; `false` nếu bài còn trong hàng đợi và đã bị gỡ ra.
`disconnect-judge`	`judge-id`, `force`	Ngắt judge (cứng hoặc gửi `disconnect`).
`disable-judge`	`judge-id`, `is-disabled`	Cập nhật cờ vô hiệu hoá trong bộ nhớ và tính lại `min_tier`.

Phía Django khi gọi `judge_submission` (`judgeapi.py:56`) làm đúng thứ tự: (0) xác định `is_pretested` và `priority` (mức 0 nếu bài thuộc kỳ thi, 1 nếu thường); (1) cập nhật bài về `QU` *có điều kiện* `exclude(status__in=('P','G'))`; nếu không dòng nào đổi thì trả `False` (chống chấm đôi); (2) xoá `SubmissionTestCase` cũ; (3) tính `banned_judges` nếu bài thuộc lượt thi trực tiếp hoặc xem; (4) nâng mức ưu tiên lên 2 (rejudge) hoặc 3 (rejudge hàng loạt) nếu cần; (5) gửi yêu cầu. Nếu gửi lỗi hoặc trả lời sai thì đặt `IE`. [S]

Bốn mức ưu tiên (`judge_priority.py`): `CONTEST_SUBMISSION_PRIORITY = 0`, `DEFAULT_PRIORITY = 1`, `REJUDGE_PRIORITY = 2`, `BATCH_REJUDGE_PRIORITY = 3`. [S]

3.4.4 Giao thức judge ↔ bridged

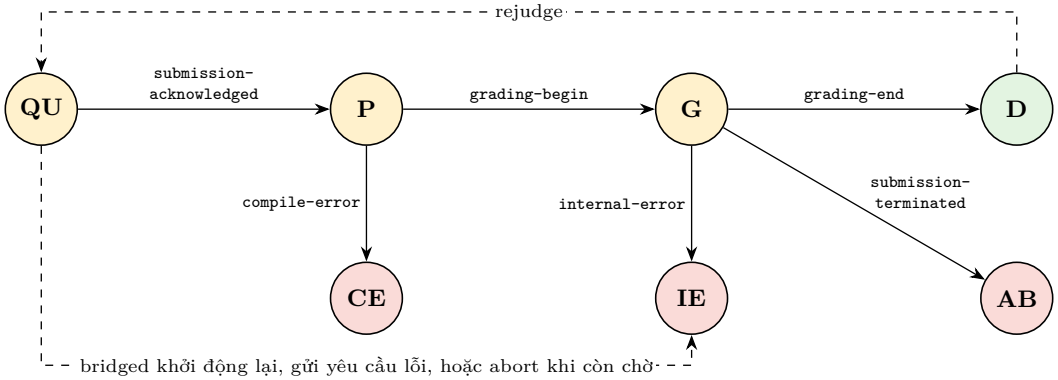
Phía judge: `dmoj/packet.py`. Phía bridged: `judge_handler.py`. Bắt tay: judge gửi `handshake` gồm `id`, `key`, `version`, `executors`, `storages`, `problems`. `bridged`: (a) **từ chối** `version == 1` (dòng 164–167); (b) tra bảng `Judge` theo `id`, so khoá bằng `hmac.compare_digest`, từ chối nếu `is_blocked`; (c) lưu `storages` (tập các `id`) và `executors`; (d) trả `handshake-success`, đăng ký vào `JudgeList`, bật luồng ping; (e) ghi bảng `Judge` (`online`, `start_time`, `last_ip`, `runtimes`, `RuntimeVersion`). Timeout socket là 15 giây trước khi bắt tay và 60 giây sau đó. [S]

Gói từ judge	Tác dụng trong `bridged`
`submission-acknowledged`	Xác nhận đúng bài; đặt `status='P'`, `judged_on=judge`; đăng sự kiện `processing`. Sai id thì đặt `IE` cho cả hai bài và ngắt judge.
`grading-begin`	`status='G'`, `is_pretested`, `current_testcase=1`, `judged_date=now`; **xoá** test case cũ.
`compile-message`	Ghi cảnh báo biên dịch vào `Submission.error`.
`compile-error`	`status='CE'`, `result='CE'`, `error=log`; giải phóng judge.
`batch-begin` / `batch-end`	Tăng số thứ tự batch (`batch_id`); bài đánh dấu `batch=True`. Các `test-case-status` nằm giữa được gán số batch này.
`test-case-status`	Mỗi gói chứa nhiều case (judge gom và gửi mỗi 0,25 giây). `bulk_create SubmissionTestCase`; dịch cờ trạng thái thành mã (bảng phía dưới). Đăng sự kiện, tối đa 5 lần mỗi 0,5 giây cho một bài.
`grading-end`	**Tính điểm và kết luận** (mục 3.4.7), `status='D'`; cập nhật điểm người dùng, thống kê bài, kỳ thi, credit; xoá cache; đăng sự kiện.
`internal-error`	`status='IE'`, `error=message`; giải phóng judge.
`submission-terminated`	`status='AB'`, `points=0`.
`ping-response`	Cập nhật độ trễ (trung bình 6 mẫu), lệch đồng hồ, `load`; ghi `Judge.ping`, `Judge.load`.
`executors`	Judge báo lại danh sách runtime; cập nhật `RuntimeVersion`.

Gói `bridged` gửi cho judge: `handshake-success`, `ping` (mỗi 10 giây), `submission-request` (kèm `time-limit`, `memory-limit`, `short-circuit`, và `meta` gồm `pretests-only`, `in-contest`, `attempt-no`, `user`, `file-only`, `file-size-limit`), `terminate-submission`, `disconnect`. [S]

Dịch cờ của judge thành mã test case. Judge gửi một số nguyên cờ (`dmoj/result.py`: `WA`=1, `RTE`=2, `TLE`=4, `MLE`=8, `IR`=16, `SC`=32, `OLE`=64). Bridge kiểm tra theo thứ tự ưu tiên `TLE` > `MLE` > `OLE` > `RTE` > `IR` > `WA` > `SC`; nếu không có cờ nào thì `AC` khi `points == total-points`, ngược lại `PAC`. [S]

3.4.5 Máy trạng thái của một bài nộp

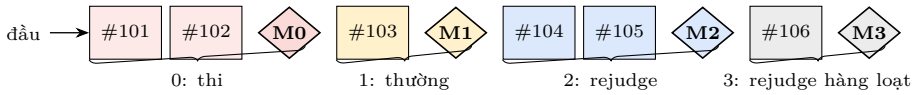


Hình 3. Các trạng thái và gói tin gây chuyển trạng thái. Nét đứt là chuyển từ phía Django hoặc khi khởi động lại `bridged`. `IE` còn xảy ra khi judge ngắt kết nối giữa lúc chấm (`judge_handler.py:on_disconnect`). Với `abort` một bài còn trong hàng đợi, `bridged` trả `judge-aborted=false` và Django đặt `AB`.

3.4.6 Lập lịch trong `JudgeList`

`judge/bridge/judge_list.py`. Khi có bài mới (`judge()`), hoặc khi một judge rảnh (`_handle_free_judge`), quy tắc như sau. [S]

1. **Tier.** Chỉ judge có `tier` bằng `min_tier` (tier nhỏ nhất trong các judge online và không bị vô hiệu hoá) được dùng. Tier lớn hơn chỉ nhận việc khi tier nhỏ hơn không còn judge nào. Đây là cơ chế dự phòng nóng.
2. **Ứng viên.** Judge không nằm trong `banned-judges`, có `storage` của bài trong tập `storages` đã khai báo, có `language` trong `executors`, và hoặc không bị vô hiệu hoá hoặc là judge được chỉ định đích danh (`judge-id`). Đây là hàm `can_judge`.
3. **Chọn.** Trong các ứng viên đang rảnh, chọn judge có `load` thấp nhất (hoà thì chọn ngẫu nhiên): `min(available, key=(load, random()))`. `load` là load-average 1 phút chia số CPU mà judge báo trong `ping-response`.
4. **Không có judge rảnh** thì chèn vào `queue`: chèn *ngay trước marker* của mức ưu tiên, nên mỗi mức là một hàng FIFO và mức nhỏ được duyệt trước.
5. **Giữ chỗ cho bài quan trọng.** Nếu mức ưu tiên ≥ 2 (rejudge) và tier hiện tại có hơn một ứng viên nhưng chỉ còn ≤ 1 judge rảnh, bài *không* được giao; judge cuối cùng được giữ cho bài thi và bài nộp thường (dòng 217–218 và hàm `should_reserve_judge`).
6. **Cảnh báo hàng đợi dài.** Khi tổng số bài mức 0 và 1 đang chờ $\geq$ `VNOJ_LONG_QUEUE_ALERT_THRESHOLD` (10) thì gọi tác vụ Celery `on_long_queue.delay(...)` để báo Discord. Lời gọi này nằm **trong vùng giữ khoá** của `JudgeList` (dòng 197–241). [I] Hệ quả được bàn ở mục 7.3.



Hình 4. `SubmissionQueue` là một danh sách liên kết đôi chứa *cả bài và 4 marker*. Bài mới chèn ngay trước marker của mức mình. Judge rảnh duyệt từ đầu nên luôn lấy bài mức nhỏ nhất, cũ nhất. Mức 2 và 3 còn chịu quy tắc "giữ chỗ".

3.4.7 Tính điểm và kết luận khi `grading-end`

Judge không tính điểm bài nộp; `bridged` tính. Judge chỉ gửi điểm từng test case. Khi nhận `grading-end`, `on_grading_end` (`judge_handler.py:372--453`) đọc lại mọi `SubmissionTestCase` của bài và: [S]

- **Thời gian** của bài là max thời gian các case; **bộ nhớ** là max bộ nhớ; tổng thời gian cộng dồn dùng để trừ *credit* tổ chức.
- **Điểm.** Case không thuộc batch: cộng `points` và `total`. Một batch: lấy **min** điểm và **max** tổng điểm của các case trong batch, rồi cộng vào. Làm tròn 3 chữ số.
- **Điểm bài** $= \text{round}\left(\frac{\text{points}}{\text{total}} \cdot P,\, 3\right)$ với $P =$ `problem.points`; nếu total $= 0$ thì 0. Nếu bài *không* cho điểm từng phần (`partial=False`) và điểm khác P thì về 0.
- **Kết luận** là case "tệ nhất" theo thứ tự `SC` < `AC` < `PAC` < `WA` < `MLE` < `TLE` < `IR` < `RTE` < `OLE`.

Sau khi lưu `status='D'`: nếu bài `is_public` và không riêng tư tổ chức thì `user.calculate_points()`; luôn gọi `problem.update_stats()`, `submission.update_contest()` (tính lại điểm trong kỳ thi và `recompute_results`), `update_credit(total_time)`, `finished_submission` (xoá cache); cuối cùng đăng sự kiện. [S]

3.4.8 Hành vi khi có sự cố

Các hành vi này đọc từ code, chưa gây lỗi thật để quan sát:

Sự cố	Kết quả theo code
`bridged` khởi động lại	Mọi bài `QU/P/G` thành `IE`; mọi judge offline cho tới khi nối lại. [S]
Judge ngắt giữa lúc chấm	`on_disconnect`: bài đang chấm thành `IE`. Judge tự thoát tiến trình (`os._exit(1)`) khi mất kết nối giữa chừng và dựa vào Docker/systemd để khởi động lại (comment trong `dmoj/packet.py:166`). [S]
Judge im lặng 60 giây	Socket timeout → `handle()` kết thúc → `on_disconnect`. Ping mỗi 10 giây nên judge sống sẽ không bị ngắt. [S]
Không còn judge nào hợp lệ	Bài ở lại hàng đợi trong RAM đến khi có judge phù hợp nối vào. [S]
Judge báo `internal-error`	Bài thành `IE`, judge được giải phóng ngay. [S]
Django không nối được `bridged`	`judge_request` ném ngoại lệ, bài thành `IE` (`judgeapi.py:105--108`). [S]
Mất MySQL	`JudgeHandler` gọi `close_if_unusable_or_obsolete` đầu mỗi lần lấy dữ liệu bài và `db.connection.close()` khi kết thúc luồng; có xử lý riêng lỗi MySQL 2006 trong `_update_ping`. Không có retry cho các lần ghi kết quả. [I]

3.5 Hệ thống sự kiện thời gian thực

Mục đích: đẩy cập nhật trạng thái bài nộp, bảng xếp hạng kỳ thi, ticket, thông báo tới trình duyệt mà không cần tải lại trang. Có ba mảnh:

1. **Phía đăng** (`judge/event_poster.py`). Có ba chế độ: tắt (`EVENT_DAEMON_USE=False`, mặc định; `post()` trả 0 ngay), AMQP (nếu có `EVENT_DAEMON_AMQP`, dùng `pika`), hoặc WebSocket tới `EVENT_DAEMON_POST`. Lỗi mạng bị nuốt (trả 0) nên sự kiện **không bao giờ làm hỏng việc chấm**. [S]
2. **Daemon** `wsevent` (`websocket/daemon.js`, Node.js với thư viện `ws`). Hai server WebSocket: một cho phía đăng (lệnh `post`, `last-msg`), một cho trình duyệt (lệnh `start-msg`, `set-filter`); thêm một HTTP long-poll ở `/channels/a|b?last=ID` (hết hạn 60 giây, trả 504). Tin nhắn nằm trong một hàng đợi **RAM**, mặc định 50 tin; id tin nhắn là `Date.now()` tăng dần. Mỗi trình duyệt lọc tối đa 5 kênh. Không lưu trữ bền vững. [S]
3. **Phía trình duyệt** (`resources/event.js`, lớp `WSEventDispatcher`). Thử WebSocket, nếu trình duyệt không hỗ trợ thì long-poll; tự nối lại sau 1 giây. Các trang dùng nó: trạng thái bài nộp, danh sách bài nộp, bảng xếp hạng kỳ thi, ticket, thông báo (`templates/submission/status.html`, `list.html`, `base.html`, `ticket/*.html`). [S]

Kênh và bảo mật. Kênh công khai: `submissions`, `tickets`. Kênh theo tài nguyên: `contest_<secret>`, `ticket-<id>`, `notification_<secret>`, và `sub_<secret>` cho từng bài nộp. `<secret>` của bài nộp là `HMAC-SHA512(EVENT_DAEMON_SUBMISSION_KEY, id)` cắt 16 ký tự hex ghép với id dạng hex (`models/submission.py: get_id_secret`). Tức là **bảo mật dựa vào việc kênh không đoán được**, và daemon không kiểm tra quyền. Khoá mặc định trong `settings.py` là chuỗi cố định công khai trong repo, nên **bắt buộc đổi** ở môi trường thật. [S]

Phát hiện: `EVENT_DAEMON_KEY` và `daemon.js` không khớp [I]

Phía đăng (`event_poster_ws.py:22--26`) gửi lệnh `auth` nếu `EVENT_DAEMON_KEY` khác `None`, và ném `EventPostingError` nếu daemon trả `error`. `daemon.js` chỉ có lệnh `post` và `last-msg` (tìm `auth` trong `websocket/` không ra kết quả). Suy ra: đặt `EVENT_DAEMON_KEY` với daemon trong repo làm mọi lần `post` ném lỗi (`EventPostingError` là `RuntimeError` nên không bị khối `except` của `post()` bắt). Cần chạy thử để chắc. Khuyến nghị: để `EVENT_DAEMON_KEY = None` và chỉ cho phép nối từ localhost/mạng nội bộ.

Nếu bỏ hẳn hệ thống này (`EVENT_DAEMON_USE=False`), trang vẫn hiện đúng trạng thái *tại lúc tải*; người dùng phải tải lại để thấy tiến triển. Không mất dữ liệu nào. [I]

3.6 Celery

Celery ở đây là **hệ thống việc nền cho tác vụ lâu hoặc định kỳ**, không nằm trên đường chấm một bài nộp thường. Cấu hình ở `dmoj/celery.py`; broker và result backend do `local_settings.py` cấp (`CELERY_BROKER_URL`, hoặc biến `*_SECRET`).

Tác vụ	Ai gọi	Ghi chú
`rejudge_problem_filter`	Admin quản lý bài (`views/problem_manage.py:95`)	Chấm lại hàng loạt; mỗi bài gọi `submission.judge(rejudge=True, batch_rejudge=True)`, tức là **đi vào `bridged` với mức 3**. Đây là chỗ duy nhất Celery chạm vào đường chấm. [S]
`rescore_problem`	Admin bài/kỳ thi	Tính lại điểm từ `case_points/case_total`, không cần judge.
`run_moss`	Admin kỳ thi	Gửi mã AC của kỳ thi tới MOSS (`pymoss`).
`prepare_contest_data`, `prepare_user_data`	Người dùng/ban tổ chức	Xuất dữ liệu (kèm trang tiến độ qua `AsyncResult`).
`send_contest_announcement`	Mô hình `ContestAnnouncement`	Gửi thông báo kỳ thi.
`restore_organization_archived_problems`	Trang tổ chức	Khôi phục bài từ kho lạnh.
`on_new_*`	Tín hiệu và view: comment, ticket, bài của tổ chức, kỳ thi	Webhook Discord. `on_new_blogpost` có định nghĩa nhưng tôi không thấy nơi gọi. [S]
`on_long_queue`	`bridged` (`judge_list.py:241`)	Cảnh báo hàng đợi dài, gộp 5 phút bằng `cache.add`.
Lịch của beat (`app.conf.beat_schedule`)		
`queue_time_stats`	mỗi ngày 00:00 UTC	Biểu đồ thời gian chờ chấm gửi Discord.
`problem_garbage_collect`	mặc định 00:00 mỗi ngày	Xoá thật các bài đã xoá mềm quá `VNOJ_PROBLEM_DELETION_GRACE_PERIOD` (7 ngày), tối đa 1 giờ/lần.
`update_hot_problems`	mỗi 10 phút	Làm mới danh sách bài "hot" (7 bài, 1 ngày).
`organization_monthly_reset`	ngày 1 hàng tháng	Chốt và đặt lại credit tổ chức.

Hai điểm cần biết khi vận hành:

- **Broker không hoàn toàn tuỳ chọn.** Nhiều view và tín hiệu gọi `.delay()` *đồng bộ trong request*: tạo comment (`signals.py:122`), tạo ticket, kỳ thi mới, thông báo kỳ thi, bài mới của tổ chức. Chỉ *worker* là tuỳ chọn; thiếu *broker* thì các thao tác này có thể lỗi hoặc chậm theo cách kombu xử lý broker không với tới được. [I]
- **Cần cả beat.** Worker không tự chạy lịch; thiếu `celery beat` thì bốn việc định kỳ ở trên không bao giờ chạy. [S]

3.7 Kho test data và đường đi của bài tập

Phía site. `StorageManager` (`judge / utils / problem _ data _ storage . py`) nạp cấu hình từ file YAML `DMOJ_STORAGE_CONFIG_PATH`: một khoá `default`, bảng `storages` (mỗi kho có `backend: filesystem` với `location`, hoặc `backend: s3` với tham số `django-storages`), và `metadata_dir` tuỳ chọn. Nếu không có file thì dùng cấu hình cũ `DMOJ_PROBLEM_DATA_ROOT` như một kho `local`; không có cả hai thì site không khởi động được. Mỗi bài có trường `Problem.storage` chọn kho. `ProblemDataStorage` là một `Storage` của Django định tuyến theo mã bài. [S]

Từ upload tới `init.yml`. Người ra đề tải một file zip và cấu hình từng test (`ProblemTestCase`). `ProblemDataCompiler.make_init()` (`judge/utils/problem_data.py`) biến chúng thành một dict rồi `yaml.safe_dump` ra `<code>/init.yml` trong kho: `archive:` là tên file zip, `test_cases:` (có batch, `dependencies`), `pretest_test_cases:`, `checker:`, `interactive:`/`signature_grader:`/`output_only:`, `file_io:`, `hints:`. Lỗi cấu hình ghi vào `ProblemData.feedback` và xoá `init.yml`. Nếu không có test nào được khai báo thì **xoá `init.yml`** để judge tự đoán tên file test. [S]

Metadata. Danh sách file trong zip và bản xem trước các test lưu ở `<metadata_dir>/<code>_metadata.json`, ghi qua file tạm rồi `os.replace`; xoá mỗi lần biên dịch lại. [S]

Phía judge. Judge đọc test trực tiếp từ đường dẫn cục bộ. Cấu hình `problem_storage_globs` liệt kê các thư mục gốc; với `judge_version: 2` mỗi mục là cặp `id` + `glob`, và **danh sách `id` được gửi trong `handshake`** (`judgeenv.py:218--231`, `packet.py:97`). Site chỉ dùng `id` để biết judge nào *có* kho nào (`can_judge`); judge **không dùng `id`** để tìm bài mà duyệt mọi gốc tìm `<gốc>/<mã bài>/init.yml` (`judgeenv.py:get_problem_root`). Với phiên bản 2, judge **không quét thư mục, không bật watchdog, không gửi danh sách bài** (`no_watchdog = True`, `get_supported_problems_and_mtimes` trả rỗng). [S]

Mỗi bài nộp, worker dựng lại `Problem` từ đầu: đọc `init.yml`, mở file zip. Nên test mới có hiệu lực ngay ở bài nộp kế tiếp, miễn file đã thấy được trên máy chấm. [S]

Phát hiện: site không đẩy test data cho judge

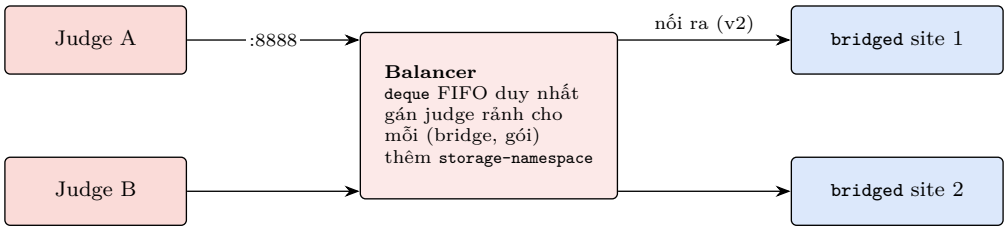
Không có đoạn mã nào trong `OJ` hay `judge-server` master chép test từ site sang judge. Site ghi vào kho của nó; judge đọc một đường dẫn cục bộ. Hai bên phải thấy **cùng dữ liệu** bằng một cơ chế *ngoài mã nguồn*: thư mục dùng chung (NFS, bind-mount cùng đĩa), rsync, hoặc kho object mount lên thư mục. Với kho S3 phía site, judge master không có mã S3 (xem mục 1), nên phải mount/đồng bộ thành thư mục. Cách VNOI đang làm [?].

Kho lạnh. `ProblemData` có `archived_size`; `judge/utils/problem_archive.py` gọi một *dịch vụ lưu trữ bên ngoài* qua HTTP (`/archive`, `/restore`, `/download`, có `Authorization: Bearer`). Thời hạn giữ `VNOJ_PROBLEM_ARCHIVE_RETENTION` chỉ dùng để *hiển thị*; theo comment trong settings, "the job owns the real schedule". [S]

Giới hạn của backend S3. `ProblemDataS3Storage._iter_keys` dùng một lần `list_objects_v2` và **ném** `RuntimeError` nếu bị cắt trang (`IsTruncated`, tức hơn 1 000 đối tượng dưới một tiền tố). `rename_folder` là *sao chép rồi xoá*, không nguyên tử. Bài có hơn 1 000 file khó đổi mã hoặc xoá trên S3. [S] [I]

3.8 Balancer (tuỳ chọn)

`judge/balancer/` cho phép **nhiều site (nhiều `bridged`) dùng chung một dàn máy chấm**. Judge không nối vào từng site mà nối vào balancer (cổng 8888, `BALANCER_JUDGE_ADDRESS`); balancer nối *ra* từng `bridged` như một judge thông thường, mỗi mục `bridges:` trong file YAML là một "judge ảo". [S]



Hình 5. Balancer đứng giữa judge và nhiều `bridged`.

Điều cần biết (`balancer.py`, `bridge_handler.py`, `judge_handler.py`):

- Hàng đợi của balancer là một `deque` **FIFO**, *không* có mức ưu tiên và *không* có tier. Ưu tiên và tier của site chỉ có tác dụng bên trong từng `bridged`. [S]
- Mỗi `bridges:` nhận **một bài một lúc**, vì site thấy nó là một judge. Muốn nhiều bài song song cho một site thì cần nhiều mục với `id` khác nhau (và nhiều bản ghi `Judge` trên site). [I]
- Đây là **nơi duy nhất** thêm trường `storage-namespace` vào gói gửi cho judge (`packet['storage-namespace'] = ...`), khớp với `storage_namespaces` trong cấu hình judge. Site VNOJ bản thân **không** gửi trường này (`judge_handler.py:submit`). [S]
- `BridgeHandler` có `judge_version` mặc định **1**, mà site từ chối phiên bản 1; cấu hình phải đặt `judge_version: 2`. [S]
- Balancer **bỏ qua** `ping-response` và `supported-problems` của judge; `executors` gửi cho các site là của *judge bắt tay gần nhất* (ghi đè). [S]
- Mất kết nối tới một `bridged` thì bài đang chạy cho bridge đó bị huỷ. [S]

Nếu chỉ có một site thì balancer là thừa. Nó đáng giá khi **nhiều site muốn chia sẻ máy chấm**; việc này [?]đang được VNOI dùng hay không.

3.9 Dịch vụ ngoài

Tất cả đều có giá trị mặc định "tắt" (`None`/`False`) trong `dmoj/settings.py`, nên site chạy được khi thiếu chúng:

Dịch vụ	Cấu hình	Dùng để
pdfoid	`DMOJ_PDF_PDFOID_URL`	Dịch đề bài HTML sang PDF (`requests.post`); có thể cache và phục vụ qua `X-Accel-Redirect`.
mathoid / texoid	`MATHOID_URL`, `TEXOID_*`	Dựng công thức toán phía server (tuỳ chọn, mặc định tắt).
camo	`DMOJ_CAMO_URL`, `_KEY`	Proxy cho ảnh nhúng từ nguồn ngoài (kiểu GitHub Camo).
Dịch vụ lưu trữ bài	`VNOJ_PROBLEM_ARCHIVE_SERVICE_URL`, `_TOKEN`	Đưa bài sang/về kho lạnh (mục 3.7).
Discord webhook	`DISCORD_WEBHOOK` (8 khoá)	Thông báo ticket/comment/bài mới/hàng đợi dài/lỗi; thống kê hàng ngày.
MOSS	`MOSS_API_KEY`	Phát hiện đạo code trong kỳ thi.
Mailgun	`MAILGUN_ACCESS_KEY`	Nhận webhook thư đến: xác minh chữ ký HMAC, rồi kích hoạt tài khoản nếu thư trả lời chứa khoá kích hoạt (`MailgunActivationView`).
reCAPTCHA, HIBP	`RECAPTCHA_*`; `api.pwnedpasswords.com`	Chống bot; kiểm tra mật khẩu đã bị lộ (`PwnedPasswordsValidator` là một trong bốn validator mật khẩu).
Đăng nhập xã hội	Google, Facebook, GitHub	OAuth2.
S3	`django-storages[s3]`	Kho test data (mục 3.7).

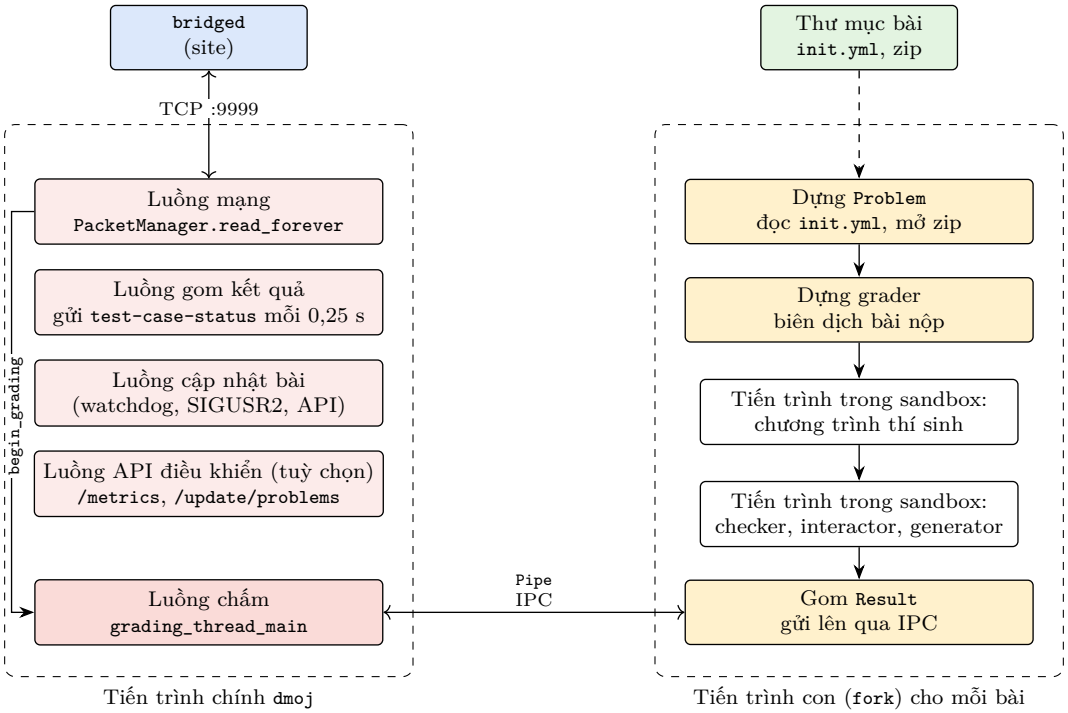
4 Judge server (`VNOI-Admin/judge-server`) chi tiết

4.1 Bản đồ mã nguồn

Thành phần	Vai trò
`dmoj/judge.py` (703 dòng)	Lớp `Judge` (điều khiển), `JudgeWorker` (tiến trình con chấm một bài), các thông điệp IPC, hàm `main()`.
`dmoj/packet.py` (406)	`PacketManager`: kết nối TCP tới `bridged`, bắt tay, đọc/ghi gói, tự nối lại.
`dmoj/judgeenv.py` (369)	Đọc cấu hình, tham số dòng lệnh, danh sách kho test, phiên bản runtime.
`dmoj/problem.py` (573)	`Problem`, `TestCase`, `BatchedTestCase`: đọc `init.yml`, ghép test, chạy generator, chọn checker.
`dmoj/graders/`	Cách chấm một test: `standard`, `interactive` (kiểu `bridged`), `communication`, `signature`, `output_only`, `custom`.
`dmoj/checkers/`	Cách so sánh đầu ra: 12 module Python và một module C nhanh (`_checker.c`).
`dmoj/executors/`	87 file: khoảng 70 executor ngôn ngữ (một file mỗi ngôn ngữ, tên viết hoa) và các lớp nền (`base_executor`, `compiled_executor`, `c_like_executor`, `python_executor`, `java_executor`, …).
`dmoj/cptbox/`	**Sandbox**: lõi Cython/C++ (`_cptbox.pyx`, `ptproc.cpp`, `helper.cpp`, `ptdebug_*.cpp` cho x86/x64/ARM/ARM64/FreeBSD) và chính sách bằng Python (`isolate.py`, `tracer.py`, `filesystem_policies.py`).
`dmoj/contrib/`	Quy ước mã thoát của checker/interactor: `default`, `testlib`, `cms`, `coci`, `peg`, `themis`.
`dmoj/cli.py`, `commands/`	`dmoj-cli`: chấm cục bộ, không cần site.
`dmoj/monitor.py`, `control.py`, `sysinfo.py`	Theo dõi thư mục bài (watchdog), API điều khiển HTTP + `/metrics`, báo load qua `ping`.
`.docker/`	Dockerfile cho 5 "tier" (`tier1`, `tier2`, `tier3`, `tiericpc`, `tiervnoj`) và script `entry`.
`testsuite/`, `dmoj/testsuite.py`	69 bộ bài/test để kiểm thử chính judge (cả executor, grader, checker).
`setup.py`	Ba lệnh: `dmoj` (judge), `dmoj-cli`, `dmoj-autoconf` (dò đường dẫn runtime).

4.2 Mô hình tiến trình

Một tiến trình `dmoj` chấm đúng một bài tại một thời điểm. `Judge.begin_grading` lấy `_grading_lock` (dòng 137) và nhả khi chấm xong; bài thứ hai gửi tới trong lúc đó sẽ chờ khoá. Phía site cũng làm đúng điều này: `JudgeHandler.working` bật khi giao bài và tắt khi `grading-end`, `compile-error`, `internal-error`, hay `submission-terminated`. Muốn chấm song song nhiều bài thì **chạy nhiều tiến trình judge với tên khác nhau**, mỗi cái là một bản ghi `Judge` trên site. [S]



Hình 6. Mô hình tiến trình của một judge. Luồng mạng nhận bài, luồng chấm sinh một tiến trình con để cô lập lỗi (kể cả crash) khỏi tiến trình điều khiển; kết quả đi ngược qua pipe, và luồng mạng/gom kết quả đẩy lên site.

Vì sao cần tiến trình con: `JudgeWorker` chấm trong một tiến trình `fork` riêng (`multiprocessing`, kiểu `fork`) nên crash hoặc rò bộ nhớ của grader/checker không làm chết judge, và có thể `SIGKILL` cả tiến trình con nếu nó treo. Luồng chấm đợi tin từ worker tối đa $\max(60, 2 \cdot \text{time_limit})$ giây; quá hạn thì giết worker và báo `internal-error`. Khi hết bài, `wait_with_timeout` chờ tối đa 60 giây rồi `SIGKILL`. [S]

Mười hai loại thông điệp IPC (`IPC.HELLO`, `BYE`, `COMPILE-ERROR`, `COMPILE-MESSAGE`, `RESULT`, `BATCH-BEGIN`, `BATCH-END`, `GRADING-BEGIN`, `GRADING-END`, `GRADING-ABORTED`, `UNHANDLED-EXCEPTION`, `REQUEST-ABORT`) ánh xạ gần 1–1 sang các gói gửi site (`judge.py:_grading_thread_main`). [S]

4.3 Cấu hình và khởi động

Cấu hình. Lệnh: `dmoj <host> [tên] [khoá] [-p 9999] [-c ~/.dmojrc] [-s TLS] [-a API-port]`. File YAML (`-c`) chứa: `id` và `key` (hoặc biến `DMOJ_JUDGE_NAME`, `DMOJ_JUDGE_KEY`); `problem_storage_globs`; `judge_version`; `runtime` (đường dẫn từng trình biên dịch, do `dmoj-autoconf` sinh); `extra_fs` (mở thêm quyền file cho từng executor); `tempdir`; `submission_cpu_affinity`; các giới hạn của compiler, generator, validator (mặc định: compiler 10 s và kích thước file nhị phân 128 MB, generator 20 s/512 MB, test tối đa 256 MB). [S]

Docker. Khi có `DMOJ_IN_DOCKER`, judge nạp `/judge-runtime-paths.yml` (do bước build sinh ra), và **API điều khiển mặc định nghe trên** `0.0.0.0:15001` (`judgeenv.py:192`); help của tham số ghi "security is left as an exercise for the reverse proxy". [S]

Trình tự khởi động (`main()`): (1) `sanity_check`: không chạy trên Windows, `cptbox` phải đã biên dịch, **không được chạy bằng root**, cảnh báo nếu BPF JIT tắt hoặc thiếu module `_checker` native; (2) `load_env`; (3) `load_executors`: với mỗi executor tìm đường dẫn, **tự kiểm tra**: biên dịch và chạy một chương trình nhỏ trong sandbox, mong nhận lại đúng `echo: Hello, World!`; executor hỏng bị bỏ; (4) `load_contrib_modules`; (5) nối `bridged`, bắt tay; (6) bật `Monitor`, API điều khiển, rồi lặp đọc gói. Danh sách executor còn lại, kèm phiên bản runtime (chạy `--version`), là trường `executors` trong `handshake`; site dựa vào đó để hiện ngôn ngữ được nhận. [S]

4.4 Một bài nộp bên trong judge

1. Luồng mạng nhận `submission-request`, **gửi** `submission-acknowledged` **trước** rồi gọi `begin_grading`.
2. Luồng chấm tạo `JudgeWorker` (= `fork`), đợi `HELLO`.
3. Trong worker, `_grade_cases()`: dựng `Problem(problem_id, time_limit, memory_limit, meta)`, đọc `init.yml`. Nếu thiếu file hoặc không có test, `InvalidInitException` dẫn tới `internal-error`.

4. Chọn `grader_class` theo khoá trong `init.yml` (mục 4.6); dựng grader, **biên dịch bài nộp trong sandbox của compiler**. Lỗi → `COMPILE-ERROR`. Có cảnh báo → `COMPILE-MESSAGE`.
5. Gửi `GRADING-BEGIN` (kèm `run_pretests_only`).
6. Duyệt các test, *phẳng hoá* batch. Với mỗi test: nếu đang "short-circuit" thì đặt `SC`; nếu không thì chấm (hoặc **dùng lại kết quả** nếu cặp (`in`, `out`) đã chấm với điểm khác 0; điểm được co giãn theo case.points · result.points/old.points).
7. Quy tắc **short-circuit** (`judge.py:500--551`): case `WA` *trong batch* → bỏ qua phần còn lại của batch; case 0 điểm mà `WA` → bỏ qua *mọi* case sau; nếu cờ `short-circuit` do site gửi là `true` (lấy từ `Problem.short_circuit`) thì *bất kỳ* `WA` nào cũng bỏ qua mọi case sau. Batch có `dependencies` bị bỏ qua nếu các batch nó phụ thuộc chưa qua hết.
8. Mỗi `Result` qua IPC → luồng chấm → `test_case_status_packet` vào hàng đợi; luồng gom gửi gộp mỗi 0,25 giây (và ngay khi `batch-*`, `grading-end`, `internal-error`).
9. Cuối cùng `GRADING-END`; luồng chấm nhả khoá, judge sẵn sàng nhận bài kế tiếp.

Nếu có yêu cầu huỷ (`terminate-submission`) trong lúc chấm, worker nhận `REQUEST-ABORT`, giết tiến trình đang chạy, gửi `GRADING-ABORTED`; kết quả đang dở bị bỏ. [S]

4.5 Định dạng bài (`init.yml`)

Site *sinh* `init.yml` (mục 3.7); judge *đọc* nó. Các khoá cấp bài và mặc định (`problem.py:ProblemConfig`):

Khoá	Mặc định	Ý nghĩa
`archive`	—	File zip chứa test, đường dẫn tương đối với thư mục bài.
`test_cases`	đoán theo tên file	Danh sách test, hoặc mô tả mẫu (`input_format`, `output_format`, `case_points`). Mỗi test: `in`, `out`, `points`, hoặc `batched` (kèm `dependencies`); có thể ghi đè `checker`, `output_limit_length`, `generator_args`.
`pretest_test_cases`	—	Chạy trước; điểm ép về 0 để thành "rào".
`checker`	`standard`	Tên checker hoặc `{name, args}`. Tên có dấu chấm được hiểu là *file Python* trong thư mục bài (xem cảnh báo ở mục 7.2).
`generator`, `generator_in`	—	Chương trình sinh input/đáp án khi chấm.
`interactive`, `communication`, `signature_grader`, `output_only`, `custom_judge`	—	Chọn grader.
`file_io`	—	Đọc/ghi qua file (`input`, `output`).
`wall_time_factor`	3	Giới hạn thời gian thực = hệ số × giới hạn CPU.
`output_limit_length`	25 165 824 (24 MiB)	Vượt → `OLE`.
`output_prefix_length`	128 (0 nếu signature)	Số byte đầu của output gửi lên site.
`points`, `binary_data`, `hints`, `unbuffered`	1, `false`, …	Điểm mặc định mỗi case; dữ liệu nhị phân; gợi ý cho executor (`unicode`, `nobigmath`); stdout không đệm.
`short_circuit`	`true`	**Không có tác dụng**: khoá được khai báo mặc định ở `problem.py:331` nhưng không nơi nào đọc; judge dùng cờ `short-circuit` trong gói từ site. [S]

Tên file test mặc định khớp biểu thức chính quy có nhóm `batch` và `case` (`DEFAULT_TEST_CASE_INPUT_PATTERN`), nên `1.in/1.out` hay `2-3.in` (batch 2, case 3) đều được nhận. Dữ liệu được **chuẩn hoá xuống dòng** (`\r \n` , `\r` → `\n`) và thêm `\n` cuối nếu thiếu, trừ khi `binary_data`. [S]

4.6 Grader: cách chấm một test

Grader	Chọn khi	Cách hoạt động
`StandardGrader`	mặc định	Chạy chương trình trong sandbox với input vào stdin (hoặc file); thu stdout/stderr; `populate_result` đặt cờ `IR/RTE/OLE/TLE/MLE`; nếu chưa cờ nào thì chạy checker.
`BridgedInteractiveGrader`	có `interactive`	Biên dịch một *interactor* (kiểu `testlib`), nối stdin/stdout hai chiều với chương trình thí sinh; mã thoát của interactor quyết định kết quả. Site chỉ sinh interactor `.cpp`, `lang: CPP20`.
`CommunicationGrader`	có `communication`	Một chương trình quản lý (`manager`) và `num_processes` tiến trình thí sinh (bản sao của cùng nhị phân) nói chuyện qua FIFO (named pipe), kiểu bài "communication" của IOI. Site không sinh cấu hình này.
`SignatureGrader`	có `signature_grader`	Ghép mã thí sinh vào một file entry và header của đề (kiểu IOI function-signature); đổi `main` của thí sinh thành tên ngẫu nhiên trừ khi `allow_main`. Hỗ trợ C/C++ và Java.
`OutputOnlyGrader`	có `output_only`	Thí sinh nộp file zip chứa đáp án; judge tải zip về (`download_source_code`) và so từng file. Ngôn ngữ `OUTPUT`.
`CustomGrader`	có `custom_judge`	Nạp một module Python từ thư mục bài và uỷ quyền cho lớp `Grader` trong đó.

Site hiện chỉ cho chọn bốn grader: `standard`, `interactive`, `signature`, `output_only` (`GRADERS` trong `models/problem_data.py`). [S]

4.7 Checker và các quy ước mã thoát

Checker	Quy tắc
`standard` (mặc định)	So theo *từ* (token), bỏ qua khác biệt khoảng trắng; cài đặt bằng C (`_checker.c`) để nhanh, có dự phòng bằng Python.
`identical`	So *từng byte*; nếu `standard` lẽ ra qua thì báo "Presentation Error".
`rstripped`	So từng dòng, bỏ khoảng trắng cuối dòng.
`linecount`, `linematches`	So từng dòng, kèm phản hồi theo dòng; `linematches` chia điểm theo dòng (`point_distribution`).
`sorted`, `unordered`	Sắp xếp rồi so (theo dòng hoặc theo từ); `unordered` đã lỗi thời.
`floats`, `floatsabs`, `floatsrel`	So số thực với sai số tuyệt đối/tương đối (`abs`, `rel`).
`easy`	So đa tập ký tự không phải khoảng trắng, không phân biệt hoa thường.
`bridged`	**Checker tuỳ chỉnh**: biên dịch một chương trình (C++/Pascal/Java; mặc định `CPP17`), chạy nó trong sandbox với `input_file output_file answer_file`, rồi đọc mã thoát theo quy ước `contrib`.

Site cho chọn: `standard`, `bridged`, `floats`, `floatsabs`, `floatsrel`, `identical`, `linecount` (`models/problem_data.py: CHECKERS`). [S]

Quy ước mã thoát (`dmoj/contrib/`). Cùng một chương trình checker có thể theo các chuẩn khác nhau, `init.yml` chọn qua `type: default` (0 = AC, 1 = WA), `testlib` (0 = AC, 1 = WA, 2 = PE, 3 = lỗi nội bộ, 7 = điểm một phần, đọc `points X` từ stderr; có chế độ coi điểm là %), `cms`, `coci`, `peg`, `themis` (đưa thư mục test/output qua stdin). Site luôn sinh `type: testlib` cho interactor và có thể kèm tham số cho checker. [S]

Bộ tiền xử lý compile cho checker/generator. `compile_with_auxiliary_files` (`utils/helper_files.py`) biên dịch các file helper *trong sandbox của compiler*; thư viện `testlib.h` được tải sẵn khi build image (`.docker/download_testlib_and_precompile`). [S]

4.8 Executor: một lớp cho mỗi ngôn ngữ

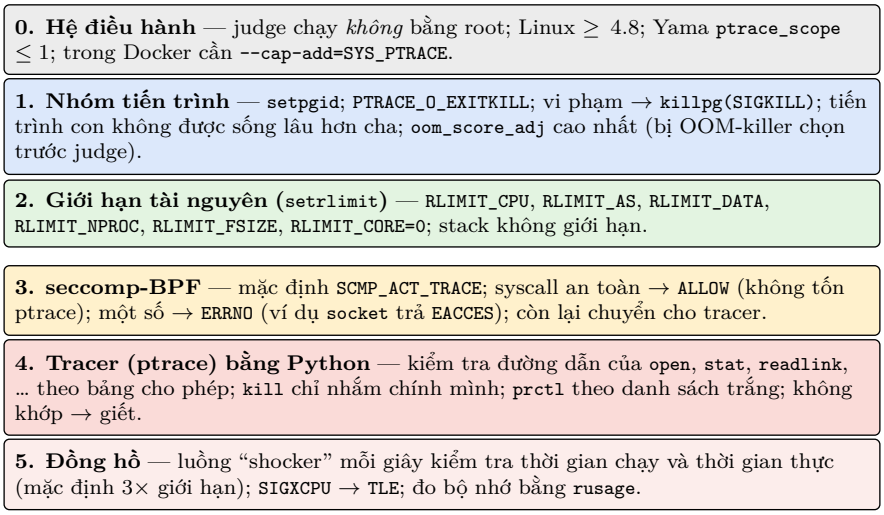
Executor trả lời bốn câu hỏi cho ngôn ngữ của nó: *biên dịch thế nào*, *chạy thế nào*, *được đọc/ghi file nào*, *được dùng syscall nào*. Cây lớp (`dmoj/executors/`):

- `BaseExecutor`: giới hạn mặc định, `BASE_FILESYSTEM` (cho đọc `/usr`, `/lib*`, `/opt`, một số file `/proc` và `/etc`; chỉ ghi `/dev/null`), phương thức `launch()` tạo `TracedPopen`, tự kiểm tra, dò phiên bản. `personality = ADDR_NO_RANDOMIZE`. [S]
- `CompiledExecutor`: biên dịch trong một sandbox *rộng hơn* (`CompilerIsolateTracer`: cho `fork`, `execve`, ghi trong thư mục tạm), có timeout (10 s), giới hạn kích thước file nhị phân, giới hạn lượng output của compiler, và **giả pty 80×1024** để compiler in màu. Có bộ nhớ đệm nhị phân tuỳ chọn. `CPP*`, `C*`, `RUST`, `GO`, `PAS`, `HASK`, … kế thừa nó. [S]
- `ScriptExecutor`: ngôn ngữ thông dịch không biên dịch (Perl, Ruby, Lua, …). `PythonExecutor` biên dịch bằng `compileall` rồi chạy `python -BS` qua một script nạp.
- `JavaExecutor`: dùng `java_sandbox.jar`, một lớp bảo vệ phía JVM. `MonoExecutor`: C#, F#, VB. `ASMExecutor`: assembly. `ShellExecutor`.

Cờ biên dịch C/C++ (`c_like_executor.py`): `-O2 -lm`, `-std=...`, `-DONLINE_JUDGE`, giới hạn số lỗi (`-fmax-errors`/`-ferror-limit`), và `-march=native` (mặc định; đổi được qua khoá `runtime`), nên nhị phân phụ thuộc CPU của máy chấm. Hai executor "đặc biệt": `BASH` và `COFFEE` bị tắt cứng (`_unsupported_executors`). [S]

4.9 Sandbox `cptbox`

Sandbox là phần *bắt buộc* của một judge: nó biến việc "chạy mã lạ" thành an toàn và đo được. `cptbox` không dùng container hay namespace; nó dùng **seccomp-BPF kết hợp ptrace** trên chính tiến trình thí sinh.



Hình 7. Các lớp bảo vệ khi chạy một chương trình thí sinh (`ptproc.cpp`, `helper.cpp`, `isolate.py`, `tracer.py`).

Vòng đời của một lần chạy (`ptproc.cpp`: `monitor`): (1) tiến trình con `fork`, đặt nhóm, gọi `PR_SET_NO_NEW_PRIVS`, nạp bộ lọc seccomp, đặt rlimit, rồi `execve`; (2) trước `execve` đầu tiên *mọi syscall được phép* để có thể in lỗi nếu khởi tạo hỏng; sau đó sandbox mới có hiệu lực; (3) tracer `wait4` cả nhóm, xử lý từng sự kiện seccomp: syscall có `handler = ALLOW` cho qua, `CALLBACK` gọi hàm Python kiểm tra, mặc định (`DISALLOW`) là **giết cả nhóm** ("safety first"); (4) kết thúc: giết mọi tiến trình con còn sót, trả mã thoát. [S]

Chính sách file. `IsolateTracer` duy trì hai "nhà tù" là cây tiền tố (`FilesystemPolicy`): đọc và ghi. Mỗi luật là `ExactFile`, `ExactDir` (chỉ liệt kê), `RecursiveDir`. Mỗi lần mở file, tracer đọc đường dẫn từ bộ nhớ tiến trình, chuẩn hoá và dùng `realpath`, so `samefile` để bắt symlink, rồi kiểm tra *cả đường dẫn gốc và đích*. Mở để ghi (`O_WRONLY`, `O_RDWR`, `O_CREAT`, `O_TRUNC`, `O_EXCL`, `O_TMPFILE`) chỉ được vào nhà tù ghi, mặc định chỉ có `/dev/null` (và thư mục tạm của bài khi compiler). [S]

Mạng. Trong sandbox *chạy bài*, `socket` và `socketcall` trả `EACCES`. Nhưng trong sandbox *biên dịch*, các syscall mạng (`socket`, `connect`, `sendto`, `recvfrom`, …) được `ALLOW` kèm comment "we don't sandbox these" (`compiler_isolate.py`). Xem mục 7.2. [S]

Thời gian và bộ nhớ. `TracedPopen` đặt: `RLIMIT_CPU` = time_limit +5 giây (chốt chặn cứng); giới hạn thời gian thực = 3× time_limit (`wall_time_factor`); `RLIMIT_AS` = bộ nhớ + `address_grace` (mặc định 64 MiB; Python 128 MiB), `RLIMIT_DATA` = bộ nhớ + `data_grace` (mặc định 0; Python 2 MiB). `MLE` được xác định khi bộ nhớ tối đa (`ru_maxrss`) vượt giới hạn; `TLE` khi thời gian chạy vượt hoặc shocker kích hoạt; `OLE` khi vượt `output_limit_length`; `IR` khi mã thoát > 0; `RTE` khi bị tín hiệu giết. [S]

Phương án nền tảng. Có mã riêng cho x86, x64, x32, ARM, ARM64 và FreeBSD x64 (`ptdebug_*.cpp`). Nhánh chưa merge `hiep/landlock` thay phần kiểm tra file bằng Landlock để giảm số lần trap ptrace; `hiep/ptraceless` thử sandbox không ptrace. Đây là hướng phát triển, chưa phải kiến trúc hiện hành. [S]

4.10 Đóng gói: các "tier" image

`.docker/Makefile` build năm image từ các image `runtimes`: `judge-tier1`, `tier2`, `tier3`, `tiericpc`, `tiervnoj`. Chúng chỉ khác *bộ runtime có sẵn*; `judge-tiervnoj` cài thêm OpenJDK 25, Go, Kotlin, Rust (qua `rustup`, có `cargo fetch` sẵn), PyPy2, scratch-run, `unzip`, `jq`. Mọi image: tải mã judge theo `TAG` (mặc định `master`) từ GitHub, tạo `venv`, `pip install -e .`, chạy `dmoj-autoconf -V` sinh `/judge-runtime-paths.yml`, biên dịch trước `testlib`. `ENTRYPOINT` là `tini` + script `entry`, có ba lệnh: `run` (→ `dmoj`), `cli` (→ `dmoj-cli`), `test` (bộ `testsuite`), và chạy bằng `setpriv` xuống người dùng `judge`. [S]

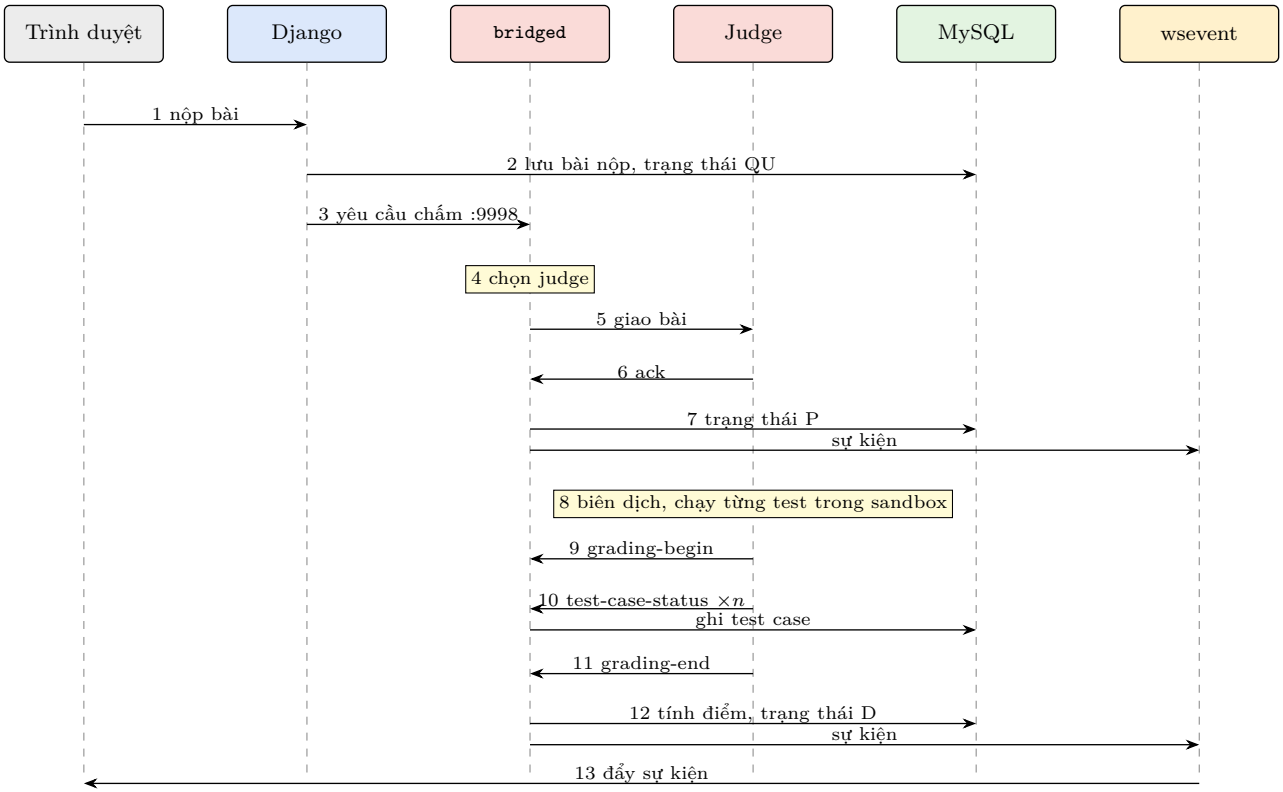
Lưu ý: Dockerfile lấy ảnh bằng nhiều lệnh `curl` tới GitHub API/PyPy/Kotlin lúc *build*, nên build phụ thuộc mạng và bị giới hạn tốc độ GitHub API (60 yêu cầu/giờ/IP khi không đăng nhập) — đã gặp khi tôi dựng `tiervnoj`; ngoài ra URL PyPy2 giờ trả `.tar.gz` thay vì `.tar.bz2` nên dòng `tar xj` hỏng. [S]

4.11 Điều khiển và quan sát

- **API điều khiển** (`control.py`, bật bằng `--api-port`): `POST /update/problems` bắt judge quét lại thư mục bài; `GET /metrics` xuất Prometheus: `dmoj_judge_up`, `dmoj_judge_problems`, `dmoj_judge_grading`, `dmoj_judge_submission_graded_total`, `dmoj_judge_case_verdicts_total` (nhãn `verdict`). **Không có xác thực.** [S]
- `dmoj-cli`: shell chấm cục bộ qua `LocalPacketManager` (mọi gói gửi site bị bỏ qua); lệnh `submit`, `rejudge`, `resubmit`, `test`, `validate`, `diff`, `problems`, `show`, `submissions`. Đây là cách thử một bài hoặc một executor mà không cần dựng site. [S]
- **Log có cấu trúc phía site**: `judge.json.bridge` ghi mỗi sự kiện (kết nối, ack, từng test case, grading-end) thành một dòng JSON, nhưng **chỉ có tác dụng nếu `LOGGING` được cấu hình** (mặc định `settings.py` không đặt). [S] [I]

5 Vòng đời một bài nộp từ đầu đến cuối

Hình 8 ghép mọi thành phần ở trên thành một chuỗi thao tác. Chữ trong hình rút gọn; bảng ngay sau đó cho tên hàm và file.



Hình 8. Sơ đồ tuần tự. Các bước 7, 10, 12 `bridged` vừa ghi MySQL vừa đăng sự kiện (nếu bật). Bước 13 chỉ xảy ra khi `wsevent` bật; nếu không, người dùng thấy kết quả khi tải lại trang.

#	Việc	Hàm và file
1	Kiểm tra quyền, tạo bài nộp	`handle_submission_post`, `judge/views/problem.py:347`
2	Ghi `Submission`, `ContestSubmission`, `SubmissionSource` trong một giao dịch	cùng hàm; sau giao dịch gọi `Submission.judge()`
3	Đặt `QU`, xoá test cũ, gửi yêu cầu	`judge_submission`, `judge/judgeapi.py:56` → `judge_request`
4	Chọn judge hoặc xếp hàng	`DjangoHandler.on_submission` → `JudgeList.judge`, `judge/bridge/judge_list.py:196`
5	Gửi cho judge kèm giới hạn đã tra DB	`JudgeHandler.submit` + `get_related_submission_data`, `judge_handler.py:192,239`
6	Judge ack rồi bắt đầu chấm	`PacketManager._receive_packet`, `dmoj/packet.py:264`; `Judge.begin_grading`, `dmoj/judge.py:131`
7	Đặt `P`, ghi `judged_on`	`on_submission_acknowledged` → `on_submission_processing`
8	Dựng `Problem`, biên dịch, chạy test trong sandbox	`JudgeWorker._grade_cases`, `dmoj/judge.py:463`; `StandardGrader.grade`, `graders/standard.py`
9	Đặt `G`	`on_grading_begin`, `judge_handler.py:357`
10	Lưu từng test, cập nhật `current_testcase`	`on_test_case`, `judge_handler.py:526`
11	Giải phóng judge, nhận bài tiếp trong hàng đợi	`_free_self` → `JudgeList.on_judge_free` → `_handle_free_judge`
12	Tính điểm, cập nhật điểm người dùng, bài, kỳ thi, credit	`on_grading_end`, `judge_handler.py:372`
13	Đăng sự kiện, trình duyệt cập nhật	`event.post`, `judge/event_poster_ws.py`; `resources/event.js`

Điểm đáng chú ý: ở bước 11, `_free_self` được gọi **ngay khi nhận `grading-end`, trước khi tính điểm** (dòng 374). Nghĩa là judge được giao bài kế tiếp trong lúc `bridged` còn đang tổng hợp điểm của bài trước ở cùng luồng `JudgeHandler`; việc tổng hợp vẫn tuần tự theo từng judge vì một luồng xử lý gói của judge đó. [S]

6 Từng thành phần: vì sao cần, thiếu thì sao, thay hoặc bỏ được không

Phần này trả lời năm câu hỏi cho từng khối, dựa trên những gì đã đọc ở các mục trước. "Thay / bỏ?" phân biệt rõ: **bỏ được** (không mất chức năng cốt lõi), **thay được** (có phương án tương đương, có thể phải sửa cấu hình hoặc code), **không thay được** (không có phương án trong source hiện tại).

A. Mặt phẳng web

A1. Reverse proxy (nginx hoặc tương đương)

Làm gì	Nhận HTTP(S), phục vụ file tĩnh và media, chuyển request động sang uWSGI, chuyển WebSocket/long-poll tới `wsevent`, và phục vụ file lớn mà Django uỷ quyền bằng `X-Accel-Redirect`.
Vì sao cần	Không để app Python phục vụ file tĩnh và giữ nhiều kết nối WebSocket dài. Có một điểm vào chung cho web và realtime. Với judge dùng TLS, cần một proxy giải mã TLS trước `bridged` (cổng 9999) vì `bridged` không biết TLS.
Nếu không có	Vẫn chạy (Django có `runserver`) nhưng mất phục vụ tĩnh hiệu quả; các tính năng dùng `X-Accel-Redirect` (tải dữ liệu người dùng, replay kỳ thi, PDF đề) không hoạt động đúng; browser không có đường tới `wsevent` cùng origin.
Thay / bỏ?	**Thay được**: Caddy, Traefik, Apache, load balancer cloud; hoặc đặt sau CDN. Đổi `uwsgi_pass` thành `proxy_pass` nếu thay uWSGI. Source `OJ` *không bắt buộc* nginx; nó chỉ có các cấu hình `*_INTERNAL` cho `X-Accel-Redirect`.
Căn cứ	`judge/utils/views.py` (đặt header `X-Accel-Redirect`), `dmoj/settings.py:266,277,287`. Cấu hình nginx cụ thể không có trong `OJ` (chỉ có ở `vnoj-docker`, bản cũ nên không dùng).

A2. Django site (uWSGI)

Làm gì	Toàn bộ nghiệp vụ: tài khoản, bài, kỳ thi (7 format), nộp bài, bảng xếp hạng, rating/PP, ticket, blog, tổ chức, admin, API, OAuth. Là bên *khởi tạo* việc chấm bài.
Vì sao cần	Là sản phẩm VNOJ. Không có nó thì không có người dùng, không có bài nộp, không có dữ liệu để chấm.
Nếu không có	Không còn hệ thống.
Thay / bỏ?	**Không bỏ được.** Lớp phục vụ WSGI thay được (uWSGI ↔ gunicorn; có sẵn `dmoj/wsgi_async.py` dùng gevent nhưng gevent không nằm trong `requirements.txt`).
Căn cứ	`judge/`, `dmoj/settings.py`, `dmoj/wsgi.py`.

B. Mặt phẳng dữ liệu

B1. MySQL / MariaDB

Làm gì	Nguồn dữ liệu duy nhất: user, bài, bài nộp, từng test, kỳ thi, bảng `Judge`. **Cả Django, Celery và `bridged` đều ghi vào đây.**
Vì sao cần	Cần lưu bền, truy vấn quan hệ, giao dịch. Là kênh trạng thái chung giữa các tiến trình: `bridged` cập nhật `Submission.status`, Django đọc để hiển thị.
Nếu không có	Không chạy được.
Thay / bỏ?	**Không bỏ được.** Thay thực tế: MariaDB. SQLite chỉ dùng cho phát triển (mặc định của `settings.py`), và code có phần dành riêng MySQL (`LOCK TABLES`, `MATCH AGAINST`, `STRAIGHT_JOIN`). PostgreSQL chưa thấy được hỗ trợ trong CI.
Căn cứ	`dmoj/settings.py:718`, `judge/dblock.py`, `judge/fulltext.py`, `.github/workflows/build.yml` (MySQL 8.0).

B2. Redis (cache, session, broker)

Làm gì	Ba vai trò độc lập: (a) *cache* Django (fragment HTML, bộ đếm, danh sách bài đã giải), (b) *session* `cached_db`, (c) *broker* (và result backend) của Celery.
Vì sao cần	Giảm tải truy vấn nặng của các trang danh sách/xếp hạng; chia sẻ cache giữa nhiều tiến trình uWSGI để việc xoá cache theo tín hiệu có hiệu lực; Celery cần một hàng đợi trung gian.
Nếu không có	Không đặt `CACHES` thì Django dùng `LocMemCache`: chạy được một tiến trình nhưng cache của các tiến trình uWSGI không đồng bộ và không được xoá đồng loạt (bài vừa sửa có thể hiện bản cũ). Không có broker thì `.delay()` có thể lỗi hoặc chậm (mục 3.6). **Đường chấm bài cũng chạm vào hai thứ này**: `on_grading_end` gọi `cache.delete_many` (`judge/caching.py`) cho mọi bài chấm xong, và `JudgeList.judge` gọi `on_long_queue.delay` khi hàng đợi dài. Nếu cache là Redis và Redis chết, lệnh xoá cache ném lỗi ở dòng 447, *sau* khi mọi cập nhật DB đã xong (điểm bài, điểm người dùng, thống kê, kỳ thi, credit): chỉ các sự kiện cuối hàm bị bỏ và cache cũ không được xoá nên trang có thể hiển thị dữ liệu cũ. [I]
Thay / bỏ?	**Thay được từng vai trò**: code chỉ gọi API cache chuẩn của Django nên đổi sang Memcached hay cache dựa trên DB được; Celery chấp nhận RabbitMQ hoặc broker khác (có sẵn `pika`). Thay đổi cần chỉnh `local_settings.py`, không sửa code.
Căn cứ	`dmoj/settings.py:775,759`, `dmoj/celery.py`, `judge/signals.py`; `requirements.txt` có `django-redis`, `pika`.

B3. Kho test data (filesystem hoặc S3)

Làm gì	Chứa mỗi bài một thư mục `<mã>/` với `init.yml`, zip test, checker/grader tuỳ chỉnh. Site *ghi* (qua `StorageManager`); judge *đọc* (qua `problem_storage_globs`).
Vì sao cần	Là hợp đồng dữ liệu giữa site và judge: judge không nhận test qua socket, mà đọc từ thư mục. Cũng cho phép nhiều kho (nhiều `id`) và gán judge theo kho.
Nếu không có	Không có thì không tải được bài; judge báo lỗi khi không tìm thấy `init.yml` (`assert root_dir is not None` trong `Problem.__init__`) dẫn tới `IE`.
Thay / bỏ?	**Thay được** backend: filesystem cục bộ, hoặc S3 cho site (judge phải mount thành thư mục). **Không có** cơ chế đồng bộ trong source: phải tự lo (NFS, rsync, bind-mount). Nhánh `microvm-api-judge` chưa merge có kho S3 phía judge.
Căn cứ	`judge/utils/problem_data_storage.py`, `judge/utils/storages/`, `dmoj/judgeenv.py`, `dmoj/problem.py`.

C. Bất đồng bộ và thời gian thực

C1. Celery worker

Làm gì	Chạy tác vụ nền: rejudge hàng loạt, rescore, MOSS, xuất dữ liệu, webhook Discord, khôi phục bài từ kho lạnh.
Vì sao cần	Các việc này lâu hoặc không nên chặn request web. Có trang tiến độ dùng `AsyncResult`.
Nếu không có	Các tác vụ trên không chạy; việc xếp hàng vẫn thành công nếu broker sống. **Nộp và chấm bài thường vẫn bình thường.** Admin không rejudge hàng loạt được.
Thay / bỏ?	**Bỏ được** ở môi trường nhỏ nếu chấp nhận mất các tính năng trên. Thay thế: chạy tác vụ đồng bộ (`CELERY_TASK_ALWAYS_EAGER`) cho thử nghiệm.
Căn cứ	`judge/tasks/`, `dmoj/celery.py`, `judge/views/problem_manage.py:95`.

C2. Celery beat

Làm gì	Bộ lập lịch: sinh các tác vụ định kỳ theo `beat_schedule`.
Vì sao cần	Worker không tự chạy lịch.
Nếu không có	Mất 4 việc định kỳ: thống kê hàng đợi, thu gom bài đã xoá, làm mới bài "hot", đặt lại credit tổ chức hàng tháng. **Bài xoá mềm sẽ không bao giờ bị xoá thật.**
Thay / bỏ?	**Bỏ được** nếu chấp nhận hậu quả. Thay thế: cron gọi `manage.py` hoặc gọi tác vụ trực tiếp.
Căn cứ	`dmoj/celery.py:25--54`.

C3. `wsevent` (Node.js) và `event_poster`

Làm gì	Hub tin nhắn trong RAM để đẩy cập nhật trạng thái bài nộp, xếp hạng kỳ thi, ticket, thông báo tới trình duyệt qua WebSocket hoặc long-poll.
Vì sao cần	Trải nghiệm trực tiếp: thấy test case chạy từng cái, bảng xếp hạng nhảy khi có AC, không phải F5.
Nếu không có	Chức năng cốt lõi không đổi. Trang hiển thị trạng thái tại lúc tải; người dùng phải tải lại. Phía đăng nuốt lỗi mạng nên không làm hỏng việc chấm.
Thay / bỏ?	**Bỏ được** (`EVENT_DAEMON_USE = False`, đã là mặc định). **Thay được**: AMQP (`EVENT_DAEMON_AMQP`, qua RabbitMQ), nhưng phía trình duyệt vẫn cần một daemon nói WebSocket/long-poll theo giao thức của `event.js`.
Căn cứ	`judge/event_poster*.py`, `websocket/daemon.js`, `resources/event.js`.

C4. Dịch vụ ngoài

Làm gì	pdfoid (PDF), mathoid/texoid (toán), camo (ảnh), dịch vụ lưu trữ bài, Discord, MOSS, Mailgun, reCAPTCHA, HIBP, đăng nhập xã hội.
Vì sao cần	Mỗi cái phục vụ một tính năng phụ trợ.
Nếu không có	Chỉ mất tính năng tương ứng (không có PDF, không có thông báo Discord, …). Mặc định trong `settings.py` đều là "tắt".
Thay / bỏ?	**Bỏ được hết**; `pwned` và `recaptcha` nên giữ cho môi trường công khai.
Căn cứ	`dmoj/settings.py`, `judge/utils/`, mục 3.9.

D. Mặt phẳng chấm bài

D1. `bridged`

Làm gì	Daemon duy nhất: nhận yêu cầu chấm từ Django, *xếp hàng theo ưu tiên*, chọn judge (tier, ngôn ngữ, kho, tải), giao việc, nhận kết quả, **tính điểm và ghi DB**, đăng sự kiện, theo dõi sức khoẻ judge bằng ping.
Vì sao cần	Tách việc chấm khỏi web: Django không giữ kết nối tới từng judge, không chờ kết quả. Judge không cần biết DB hay Django. Mọi quy tắc phân phối (ưu tiên thi, giữ chỗ, dự phòng nóng) nằm một chỗ.
Nếu không có	Bài nộp được tạo nhưng **không có gì chấm**: Django gửi yêu cầu thất bại và đặt `IE`. Judge không có chỗ để nối.
Thay / bỏ?	**Không thay được** trong source hiện tại. Cũng là **điểm đơn lẻ**: một tiến trình, hàng đợi trong RAM, khởi động lại mất toàn bộ hàng đợi và các bài đang chấm thành `IE`. Phương án chịu lỗi duy nhất có sẵn là **tier** của judge (dự phòng *judge*, không phải dự phòng `bridged`) và (phần nào) balancer.
Căn cứ	`judge/bridge/`, mục 3.4.

D2. Judge server (tiến trình `dmoj`)

Làm gì	Nối tới `bridged`, nhận bài, tải `init.yml`, biên dịch, chạy từng test trong sandbox, chạy checker, báo kết quả.
Vì sao cần	Chạy mã lạ cần một tiến trình riêng, có quyền hạn khác, đặt được ở máy riêng. Một judge chấm một bài một lúc; số judge quyết định thông lượng.
Nếu không có	Không có judge: hàng đợi tích tụ; bài không bao giờ được chấm. Không có judge cho một *ngôn ngữ* hay *kho*: bài đó bị xếp hàng mãi (không có thông báo lỗi riêng), trong khi `Problem.usable_languages` sẽ ẩn ngôn ngữ đó khỏi form nộp.
Thay / bỏ?	**Không bỏ được.** Thay thế: một triển khai khác nói cùng giao thức (phiên bản 2) cũng dùng được; nhánh `microvm-api-judge` chưa merge là hướng làm judge chạy qua HTTP API.
Căn cứ	`dmoj/judge.py`, `dmoj/packet.py`, mục 4.

D3. Sandbox `cptbox`

Làm gì	Cô lập chương trình thí sinh: seccomp + ptrace + rlimit + chính sách file; đo thời gian, bộ nhớ, đầu ra.
Vì sao cần	Không thể chạy mã của người lạ mà không có nó: có thể đọc file, mở mạng, chiếm CPU/RAM, fork-bomb. Cũng là nguồn các phép đo TLE/MLE/OLE.
Nếu không có	Thí sinh có toàn quyền như người dùng chạy judge; không có phép đo.
Thay / bỏ?	**Không bỏ được.** Thay thế: sandbox khác như `isolate` hoặc Landlock + seccomp-notify (các nhánh `hiep/landlock`, `hiep/ptraceless` chưa merge). Đổi sandbox là thay `dmoj/cptbox/`, không đụng giao thức.
Căn cứ	`dmoj/cptbox/`, mục 4.9.

D4. Executor và runtime (image Docker)

Làm gì	Biết cách biên dịch và chạy từng ngôn ngữ; image Docker cung cấp compiler/interpreter.
Vì sao cần	Mỗi ngôn ngữ cần cờ, quyền file và syscall riêng.
Nếu không có	Judge không có executor nào cho ngôn ngữ đó → không khai báo trong `handshake` → site không giao bài ngôn ngữ đó cho nó.
Thay / bỏ?	**Bỏ được từng ngôn ngữ** (`--only-executors`/`--exclude-executors`). **Thay được** bộ runtime bằng image khác; các "tier" chỉ là bộ ngôn ngữ khác nhau.
Căn cứ	`dmoj/executors/`, `.docker/`, mục 4.10.

D5. Grader và checker

Làm gì	Grader quyết định *cách* chấm (chuẩn, tương tác, hàm, chỉ output); checker quyết định *so sánh* đầu ra thế nào.
Vì sao cần	Nhiều đề không so khớp từng từ: đáp án nhiều cách (checker tuỳ chỉnh), số thực (`floats`), tương tác.
Nếu không có	Chỉ còn `StandardGrader` + `standard` checker: đủ cho bài "in ra đáp án duy nhất".
Thay / bỏ?	**Bỏ được** các bản chuyên biệt nếu chỉ có bài đơn giản; không bỏ được grader chuẩn và `standard`. Site chỉ cho chọn 4 grader và 7 checker.
Căn cứ	`dmoj/graders/`, `dmoj/checkers/`, `dmoj/contrib/`.

D6. Balancer

Làm gì	Cho nhiều `bridged` (nhiều site) dùng chung một dàn judge; thêm `storage-namespace`.
Vì sao cần	Một dàn máy chấm đắt; nhiều site muốn tận dụng chung.
Nếu không có	Mỗi site tự có judge của mình; không ảnh hưởng chức năng.
Thay / bỏ?	**Bỏ được** và nên bỏ nếu chỉ một site. Không thay thế sẵn có. Lưu ý các giới hạn: FIFO, không ưu tiên, một bài một lúc cho mỗi mục `bridges` (mục 3.8).
Căn cứ	`judge/balancer/`.

D7. `dmoj-cli`, `testsuite`, `dmoj-autoconf`

Làm gì	Công cụ phát triển/vận hành: chấm cục bộ không cần site, kiểm thử judge, dò đường dẫn compiler.
Vì sao cần	Thử một bài hay một executor nhanh; kiểm tra bản build.
Nếu không có	Không ảnh hưởng chức năng chạy thật, nhưng `dmoj-autoconf` được dùng **khi build image** để sinh `/judge-runtime-paths.yml`, nên judge trong Docker cần nó.
Thay / bỏ?	`dmoj-cli`/`testsuite` **bỏ được**; `dmoj-autoconf` cần khi build.
Căn cứ	`dmoj/cli.py`, `dmoj/testsuite.py`, `dmoj/executors/autoconfig.py`.

6.1 Bảng quyết định tổng hợp

Thành phần	Mức cần	Thay bằng	Có thể bỏ khi
Django + uWSGI	bắt buộc	gunicorn	—
MySQL/MariaDB	bắt buộc	MariaDB	— (SQLite chỉ để dev)
`bridged`	bắt buộc để chấm	(không)	—
Judge ≥ 1	bắt buộc để chấm	(giao thức v2)	—
`cptbox`	bắt buộc	sandbox khác (chưa merge)	—
Kho test data	bắt buộc	FS $\leftrightarrow$ S3 (site)	—
Reverse proxy	rất nên có	Caddy, Traefik, CDN	chạy thử nghiệm
Cache chung	rất nên có	Memcached, DB cache	một tiến trình duy nhất
Celery broker	cần cho vài thao tác web	RabbitMQ	không dùng các thao tác đó
Celery worker	tuỳ chọn	chạy đồng bộ	không rejudge hàng loạt, không MOSS
Celery beat	tuỳ chọn	cron	chấp nhận không dọn bài xoá
`wsevent`	tuỳ chọn	AMQP (nhưng cần daemon cho trình duyệt)	không cần cập nhật trực tiếp
Balancer	tuỳ chọn	(không)	chỉ một site
Dịch vụ ngoài	tuỳ chọn	—	không cần tính năng tương ứng

7 Sự cố, bảo mật và các phát hiện từ việc đọc code

7.1 Ma trận sự cố

Mỗi dòng trả lời: nếu *một* khối ngừng hoạt động thì người dùng và hệ thống thấy gì. Tất cả suy ra từ code ([S] hoặc [I] như đã ghi ở các mục trước), chưa gây sự cố thật.

Khối ngừng	Xem trang	Nộp bài	Chấm bài
Django/uWSGI	Không	Không	Judge và `bridged` chạy tiếp; bài đang chấm *vẫn hoàn tất và ghi DB*.
MySQL	Không	Không	`bridged` không ghi được kết quả; judge vẫn chạy nhưng kết quả mất hoặc `IE`. [I]
`bridged`	Có	Lỗi: bài thành `IE` ngay	Mất toàn bộ hàng đợi RAM; bài `QU/P/G` thành `IE` khi `bridged` lên lại; judge *rảnh* tự nối lại (4 giây, tăng dần tới 60 giây); judge *đang chấm* gặp lỗi gửi gói thì thoát tiến trình (`os._exit(1)`) và cần Docker/systemd khởi động lại.
Một judge	Có	Có	Bài đang chấm trên nó thành `IE`; các judge khác chạy tiếp.
Mọi judge hợp lệ cho (ngôn ngữ, kho)	Có	Có	Bài ở `QU` trong RAM cho tới khi có judge.
Cache dùng chung (Redis)	Nhiều trang lỗi [I]	Lỗi [I]	Điểm vẫn lưu, mất sự kiện cuối và việc xoá cache. [I]
Celery worker	Có	Có	Không ảnh hưởng chấm thường; rejudge hàng loạt không chạy.
Celery broker	Một số thao tác lỗi [I]	Có	Có, trừ `on_long_queue.delay` có thể chặn [I] (mục 7.3).
Celery beat	Có	Có	Có; mất 4 việc định kỳ.
`wsevent`	Có (không tự cập nhật)	Có	Có.
Kho test mất trên judge	Có	Có	Mỗi bài của đề đó thành `IE` (không tìm thấy `init.yml`). [I]
Balancer	Có	Có	Judge sau balancer rời site; bài đang chạy qua nó bị huỷ.

7.2 Bảo mật và cấu hình mặc định

Phía site. `dmoj/settings.py` là cấu hình *phát triển*; `local_settings.py` phải ghi đè: [S]

- `SECRET_KEY`, ba khoá `EVENT_DAEMON_*_KEY` (`SUBMISSION`, `CONTEST`, `TICKET`, `NOTIFICATION`) đều là chuỗi cố định công khai trong repo. Khoá `SUBMISSION` quyết định tên kênh sự kiện của từng bài nộp (mục 3.5); để mặc định thì ai cũng tính ra được kênh.
- `DEBUG = True`, `ALLOWED_HOSTS = []`, `DATABASES` là SQLite, `CACHES = {}`.
- `GLOBAL_API_KEY` mặc định là `test-api-key-123`; nó bảo vệ API đồng bộ kỳ thi: chỉ có hiệu lực khi `VNOJ_ENABLE_SYNC_API = True`.
- `bridged` mặc định nghe `localhost`; muốn judge ở máy khác nối vào phải đổi địa chỉ và **tự bảo vệ** cổng 9999: xác thực chỉ là cặp `id` + `key`, `Judge.auth_key` lưu **dạng thô** trong DB, giao thức không mã hoá. Nên đặt sau proxy TLS (judge có `--secure`) hoặc VPN, và cổng 9998 (Django → `bridged`, *không xác thực*) chỉ nên mở cho localhost.
- Phản hồi `EventPostingError` khi đặt `EVENT_DAEMON_KEY` (mục 3.5).

Phía judge. Những điểm sau đọc trực tiếp từ code:

- **Mạng trong giai đoạn biên dịch không bị chặn bởi sandbox.** `CompilerIsolateTracer` cho phép `socket`, `connect`, `sendto`, `recvfrom` ("we don't sandbox these"), trong khi sandbox *chạy bài* trả `EACCES` cho `socket`. Nếu trình biên dịch của một ngôn ngữ có thể chạy mã do thí sinh viết ngay lúc biên dịch, mã đó có mạng chừng nào container có mạng ra ngoài. Biện pháp thực tế: chạy judge trong mạng không có đường ra. [S] (hậu quả [I])
- **Mã trong thư mục bài được thực thi *không* qua sandbox.** Checker có tên chứa dấu chấm được nạp bằng `exec(compile(...))` trong tiến trình worker (`utils/module.py`, `problem.py:load_checker`); `custom_judge`

cũng vậy. Site *không* sinh các khoá này (chỉ cho 7 checker, trong đó `bridged` chạy trong sandbox), nên chỉ rủi ro khi thư mục bài do người khác viết tay. Ai ghi được vào kho test data thì chạy được Python trên máy chấm. [S]

- **API điều khiển không xác thực** (`POST /update/problems`, `GET /metrics`) và trong Docker **mặc định nghe** `0.0.0.0:15001` nếu không đặt `--api-port` (`judgeenv.py:192`). Cần chặn bằng firewall hoặc proxy. [S]
- Judge **không được chạy bằng root** (`sanity_check`); script `entry` hạ quyền xuống người dùng `judge` bằng `setpriv`. Cần `--cap-add=SYS_PTRACE` trong Docker: một capability mạnh; nên cân nhắc cùng với `seccomp`/AppArmor của container và mạng cách ly. [S]
- `Judge.is_blocked` chặn judge theo tên ngay lúc bắt tay; `contest.banned_judges` loại judge khỏi kỳ thi. [S]

7.3 Các phát hiện

Những chỗ tôi thấy khác với kỳ vọng hoặc dễ gây nhầm khi đọc source, xếp theo mức đáng chú ý. Mỗi mục kèm độ chắc chắn.

F1. Bộ hẹn giờ 20 giây "judge không ack" không bao giờ chạy [S]

`JudgeHandler.submit` tạo `threading.Timer(20, self._kill_if_no_response)` (`judge_handler.py:245`) và `on_submission_acknowledged` gọi `.cancel()`, nhưng **không có lời gọi `.start()` nào** trong `judge/bridge/` hay `judge/balancer/` (grep `\.start()` chỉ ra các luồng khác). Ý đồ của mã là ngắt judge nào không xác nhận trong 20 giây. Thực tế judge "câm" chỉ bị phát hiện qua timeout socket 60 giây (`on_timeout` → `handle()` kết thúc → `on_disconnect`). Hậu quả nhỏ vì ping mỗi 10 giây, nhưng đây là mã chết.

F2. Khoá `short_circuit` trong `init.yml` không có tác dụng [S]

`problem.py:331` khai báo mặc định `'short_circuit': True`, nhưng `judge.py:501` lấy giá trị từ `self.submission.short_circuit`, tức cờ do site gửi (`Problem.short_circuit`). Không nơi nào đọc `config.short_circuit`.

F3. Gọi Celery trong vùng giữ khoá của `JudgeList` [I]

`JudgeList.judge` gọi `on_long_queue.delay(...)` *bên trong* `with self.lock` (`judge_list.py:197--241`). Khi hàng đợi mức 0–1 từ 10 bài trở lên và broker không với tới được, kombu sẽ thử kết nối lại và ném lỗi; trong thời gian đó khoá bị giữ nên `register`, `on_judge_free`, `remove` của mọi judge phải chờ. Lỗi ném ra từ `judge()` làm `DjangoHandler` trả `bad-request`, và Django (`judgeapi.py:110`) đặt bài là `IE` *dù bài đã nằm trong hàng đợi*; khi judge nhận bài, trạng thái sẽ tự sửa qua `P/G/D`. Cần chạy thử với broker tắt để xác nhận.

F4. Judge bị vô hiệu hoá có thể nhận một bài ngay khi nối lại [I]

Trong `on_handshake`, `self.judges.register(self)` (dòng 180) chạy *trước* `self._connected()` (dòng 182) mới đọc `is_disabled` từ DB. `is_disabled` khởi tạo `False` (dòng 70), nên lúc `register` → `_handle_free_judge`, `can_judge` cho qua và `min_tier` cũng tính judge đó; nếu hàng đợi có bài hợp lệ, bài được giao đi. Cờ chỉ có hiệu lực từ lúc đó trở đi. Cần chạy thử.

F5. `EVENT_DAEMON_KEY` và `daemon.js` không khớp [I]

Đã nêu ở mục 3.5.

F6. Lệnh `/update/problems` của judge gây lỗi log ở site [I]

Judge v2 không có watchdog nên không tự gửi danh sách bài; nhưng nếu gọi `POST /update/problems` hay gửi `SIGUSR2`, luồng cập nhật vẫn gửi gói `supported-problems` (với danh sách rỗng). `bridged` **không có handler** cho gói này (chỉ balancer có `ignore_packet`) nên rơi vào `on_malformed`, ghi log lỗi. Vô hại về chức năng.

F7. Site không đẩy test data; cũng không biết judge nào đã có bài nào [S]

Site không còn xử lý `supported-problems` và bảng `Judge` không có quan hệ tới bài. Phân phối theo *kho* (`id`), không theo *bài*. Nếu một bài nằm trong kho `A` mà một judge khai báo kho `A` nhưng thư mục của nó chưa có bài đó, judge nhận bài rồi báo `IE`. Đây là thiết kế: trách nhiệm đồng bộ thuộc về người vận hành.

F8. Balancer cần `judge_version: 2`; `storage-namespace` chỉ do balancer thêm [S]

Xem mục 3.8. `BridgeHandler` mặc định `judge_version=1` mà site từ chối.

F9. Backend S3 có giới hạn 1 000 đối tượng cho đổi tên/xoá thư mục [S]

Xem mục 3.7: `_iter_keys` ném `RuntimeError` khi `IsTruncated`.

F10. Nhị phân biên dịch bằng `-march=native` [S]

Cờ mặc định của executor C/C++ (`c_like_executor.py:105`). Chấm trên nhiều máy CPU khác nhau cho cùng mã vẫn đúng (mỗi máy tự biên dịch), nhưng thời gian chạy có thể lệch giữa các máy; xem điều này khi hai judge có CPU khác thế hệ.

7.4 Những điều chưa xác minh

Cần quan sát hệ thống chạy thật hoặc hỏi người vận hành:

1. **Cấu trúc triển khai của VNOI**: bao nhiêu máy chấm, có dùng tier và balancer không, `bridged` đặt cùng máy với web hay riêng. [?]
2. **Cách đồng bộ test data** từ site tới judge (NFS, rsync, mount S3, …). [?]
3. Nội dung `websocket/config.js` (không có trong repo), cổng thật của `wsevent`, và cấu hình nginx. [?]
4. Cấu hình `LOGGING` (nơi các logger `judge.bridge`, `judge.json.bridge`, `judge.celery` trỏ tới). [?]
5. Broker Celery đang dùng (Redis hay RabbitMQ) và backend cache. [?]
6. Mọi mục gắn [I]: cần chạy thử bằng cách gây sự cố có kiểm soát (tắt broker, tắt Redis, nối lại judge bị vô hiệu hoá).

A Phụ lục

A.1 Thuật ngữ và mã trạng thái

Mã	Ý nghĩa
`QU`, `P`, `G`, `D`	Trạng thái bài nộp: Queued (chờ), Processing (judge đã nhận), Grading (đang chấm), Done (đã xong).
`CE`, `IE`, `AB`	Compile Error, Internal Error (lỗi phía máy chấm), Aborted (bị huỷ). Vừa là trạng thái vừa là kết quả.
`AC`, `PAC`, `WA`	Accepted, Partially Accepted, Wrong Answer.
`TLE`, `MLE`, `OLE`	Quá thời gian, quá bộ nhớ, quá giới hạn output.
`RTE`, `IR`	Runtime Error (bị tín hiệu giết); Invalid Return (mã thoát khác 0).
`SC`	Short-Circuited: bị bỏ qua vì một test trước đó đã sai.
Tier	Mức dự phòng của judge; chỉ tier nhỏ nhất đang online được dùng.
Kho (storage)	Nơi chứa dữ liệu bài, có `id`; judge khai báo nó có kho nào.
Batch	Nhóm test chung một điểm (min điểm các test trong nhóm); có thể phụ thuộc batch trước.
Pretest	Test chạy trước, điểm 0, dùng làm "rào" sơ bộ.

A.2 Cách kiểm tra lại các khẳng định

Mọi khẳng định có nhãn [S] có thể kiểm tra bằng các lệnh sau (đường dẫn tương đối với thư mục gốc từng repo):

Khẳng định	Lệnh
Mốc commit	`git log -1`
Timer không được start (F1)	`grep -n "Timer|\.start()" judge/bridge/*.py judge/balancer/*.py`
`short_circuit` không đọc (F2)	`grep -rn short_circuit dmoj --include=*.py`
Không có mã S3 phía judge	`grep -rIiw "s3|boto" dmoj --include=*.py`
Judge từ chối phiên bản 1	`grep -n "version" judge/bridge/judge_handler.py`
Handler của gói judge	`sed -n 46,61p judge/bridge/judge_handler.py`
Mạng trong sandbox biên dịch	`grep -n "socket|connect" dmoj/cptbox/compiler_isolate.py`
Daemon `wsevent` không có `auth`	`grep -n "auth" websocket/*.js` (không có kết quả)
Nơi gọi `.delay()`	`grep -rn "\.delay(" judge --include=*.py`
Nhánh chưa merge	`git log master..origin/<nhánh>`

A.3 Các file đáng đọc nhất

Nếu chỉ có thời gian đọc mười file, đây là thứ tự khuyến nghị:

1. `OJ/judge/bridge/judge_list.py` — lập lịch, ưu tiên, tier (241 dòng).
2. `OJ/judge/bridge/judge_handler.py` — giao thức judge và tính điểm (668 dòng).
3. `OJ/judge/judgeapi.py` — phía Django của giao thức (137 dòng).
4. `OJ/judge/bridge/daemon.py` — khởi tạo và hành vi khi khởi động lại (50 dòng).
5. `judge-server/dmoj/judge.py` — mô hình tiến trình và `_grade_cases` (703 dòng).
6. `judge-server/dmoj/packet.py` — giao thức phía judge (406 dòng).
7. `judge-server/dmoj/problem.py` — `init.yml` và test case (573 dòng).
8. `judge-server/dmoj/cptbox/ptproc.cpp` và `isolate.py` — sandbox.
9. `OJ/judge/utils/problem_data_storage.py` và `problem_data.py` — kho và `init.yml`.
10. `OJ/dmoj/settings.py` — mọi cấu hình mặc định và cờ.